Algorithms

DFS & BFS

SOME SLIDES ARE TAKEN FROM ALGORITHMS COURSE, TECHNION

Last Week

- Trees
- DFS of undirected graphs

This Week

- DFS of directed graphs
- BFS

DFS of a Directed Graph

Algorithm 1 DFS(G)

For each $v \in V$:
    if v is unmarked then
        DFS-Visit(G,v)

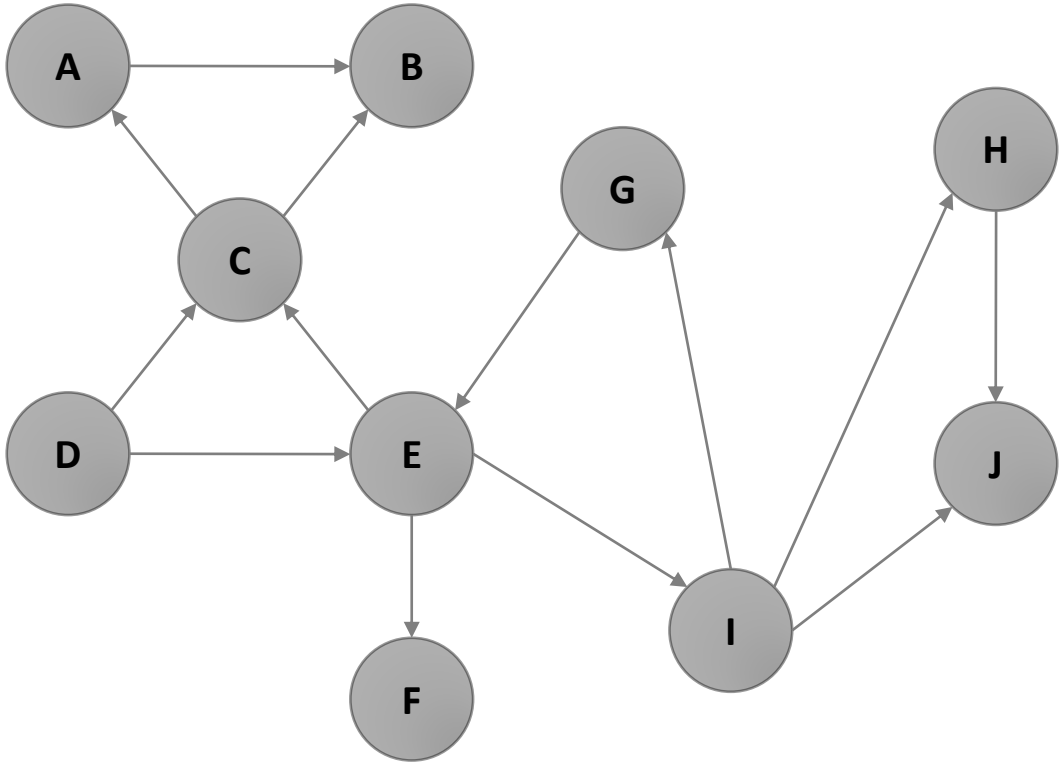
Algorithm 2 DFS-Visit(G,v)

mark v
For each u s.t. $(v, u) \in E$:
    if u is unmarked then
        add (v, u) to the spanning tree
        DFS-Visit(G,u)

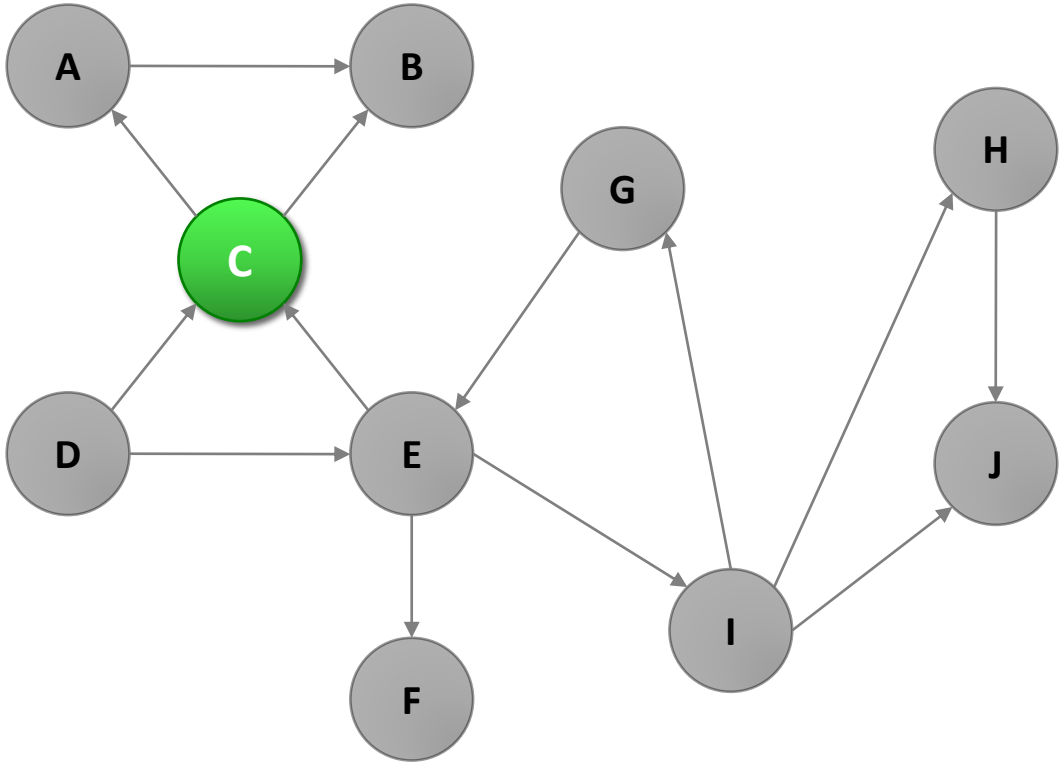
Note: can result in a forest and not a tree

Complexity? **O(n+m)**

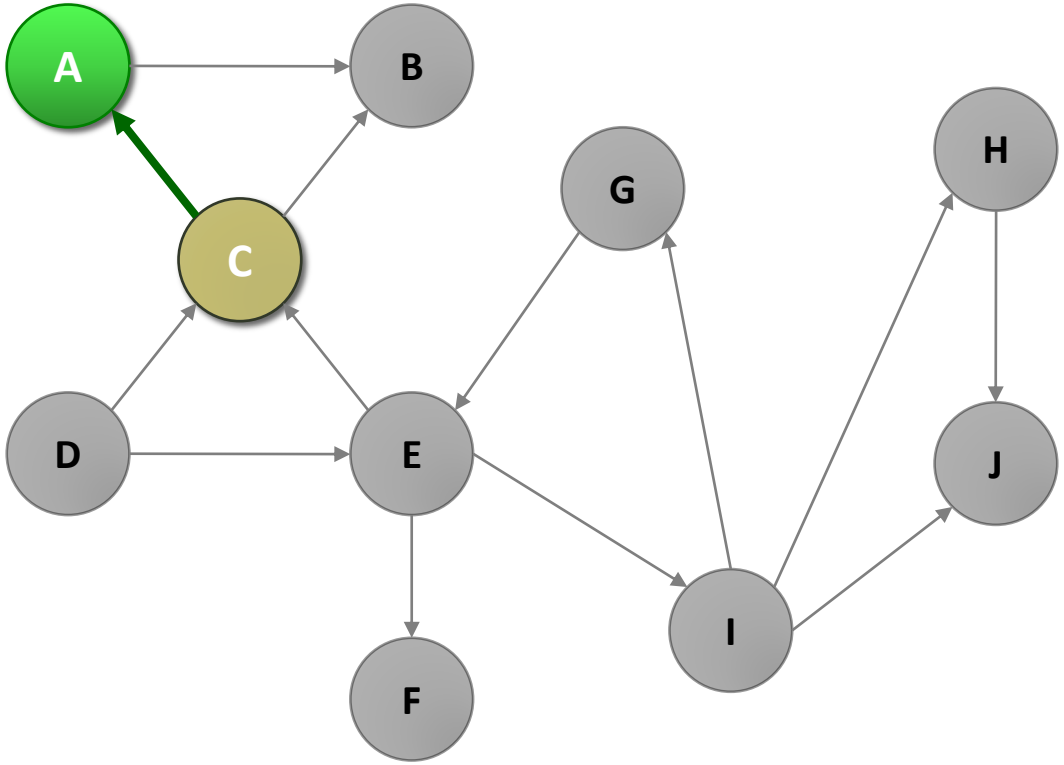
DFS



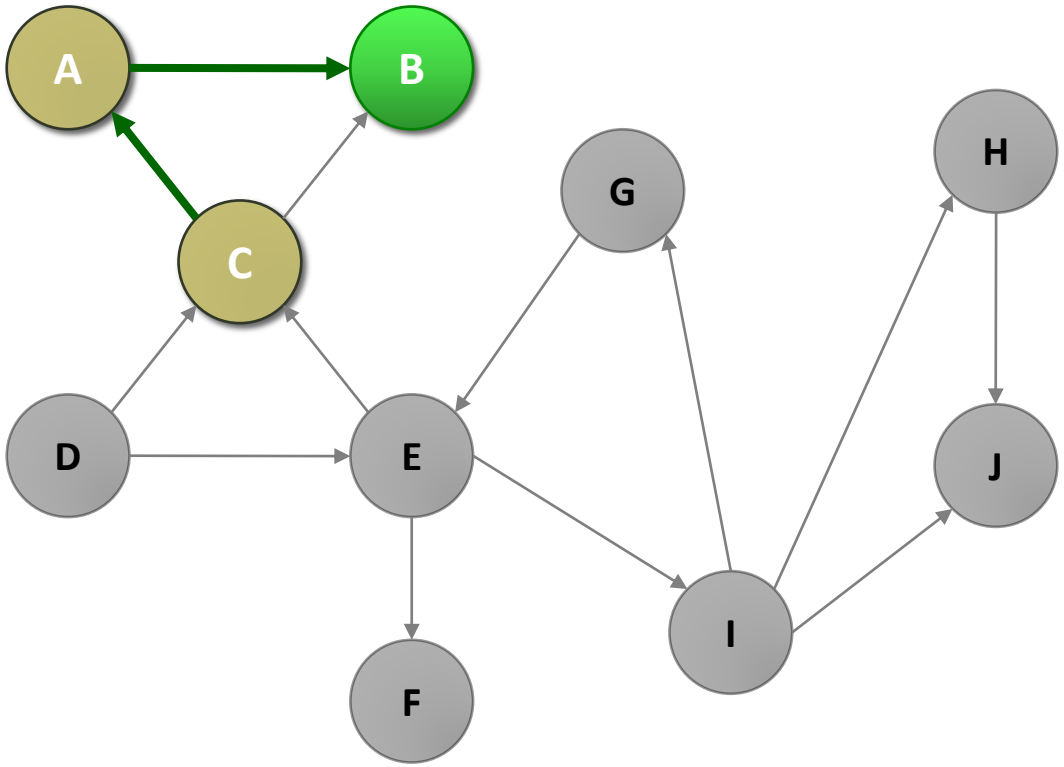
DFS



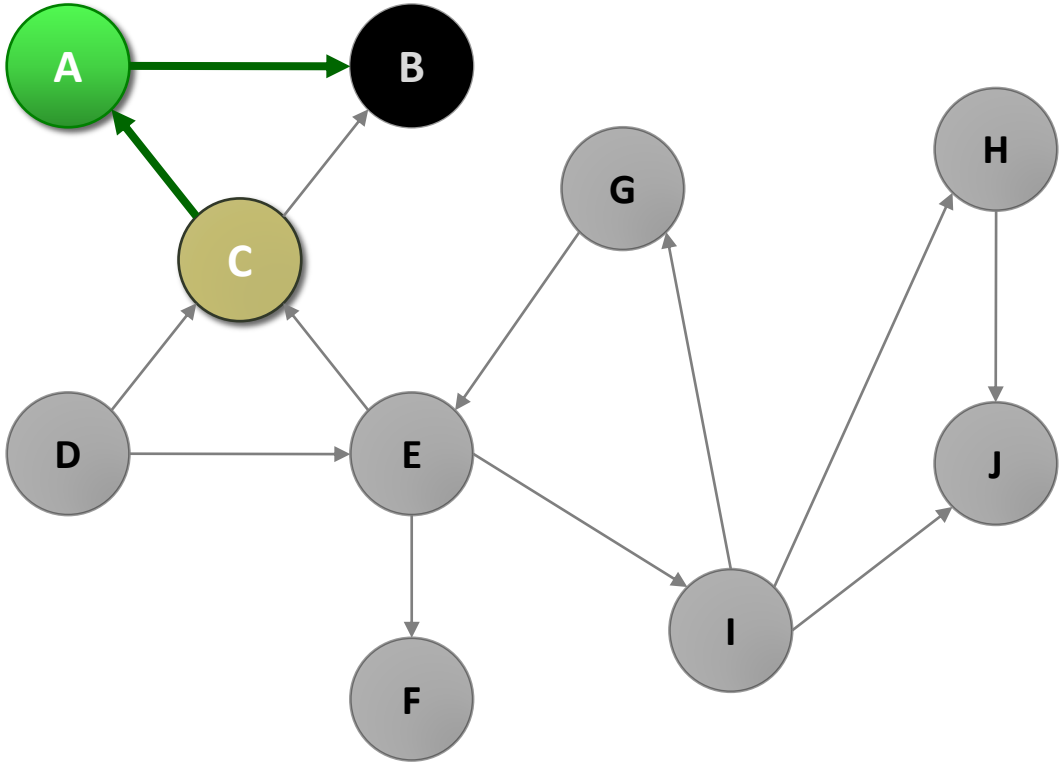
DFS



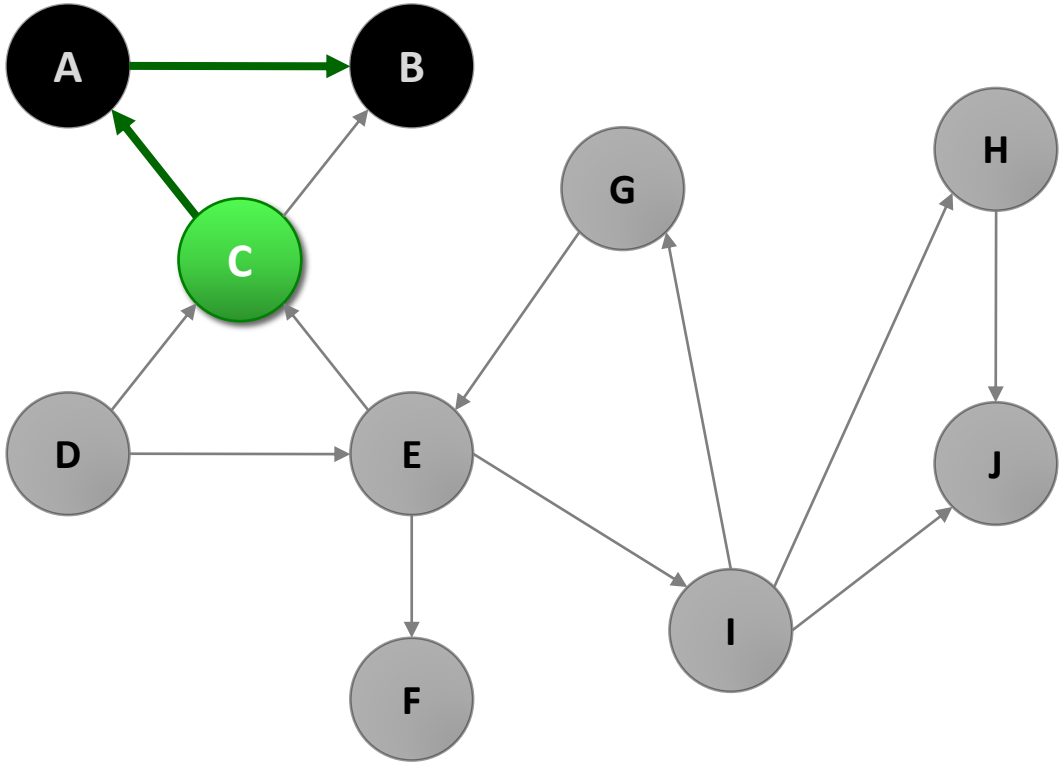
DFS



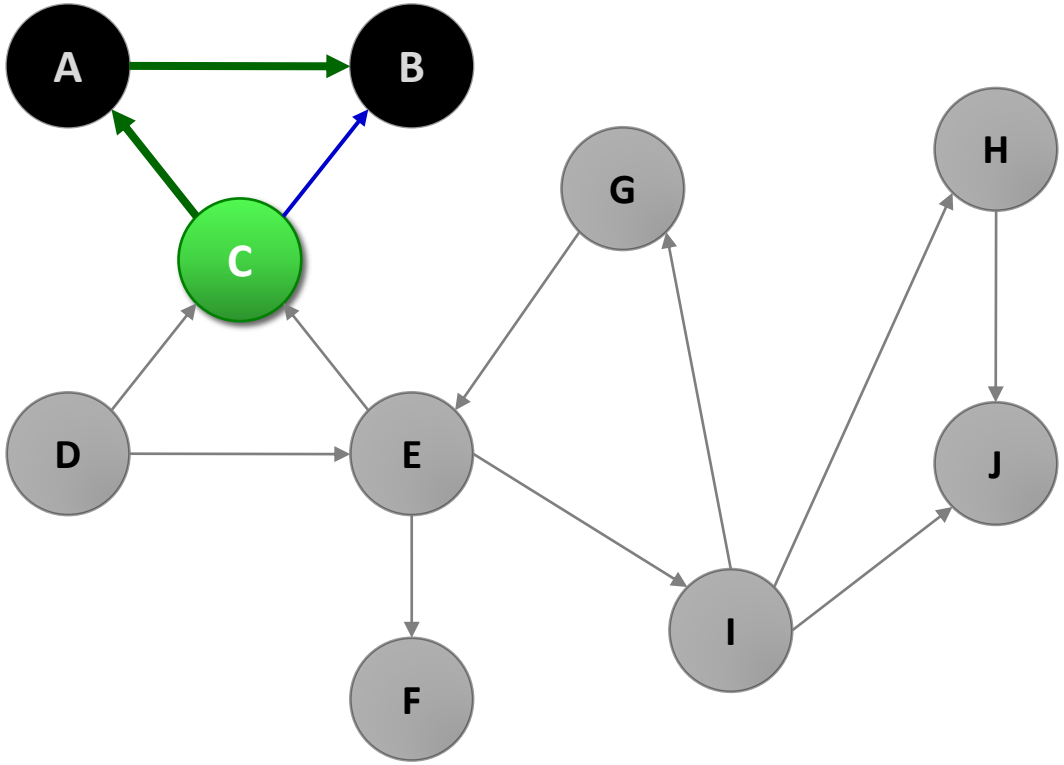
DFS



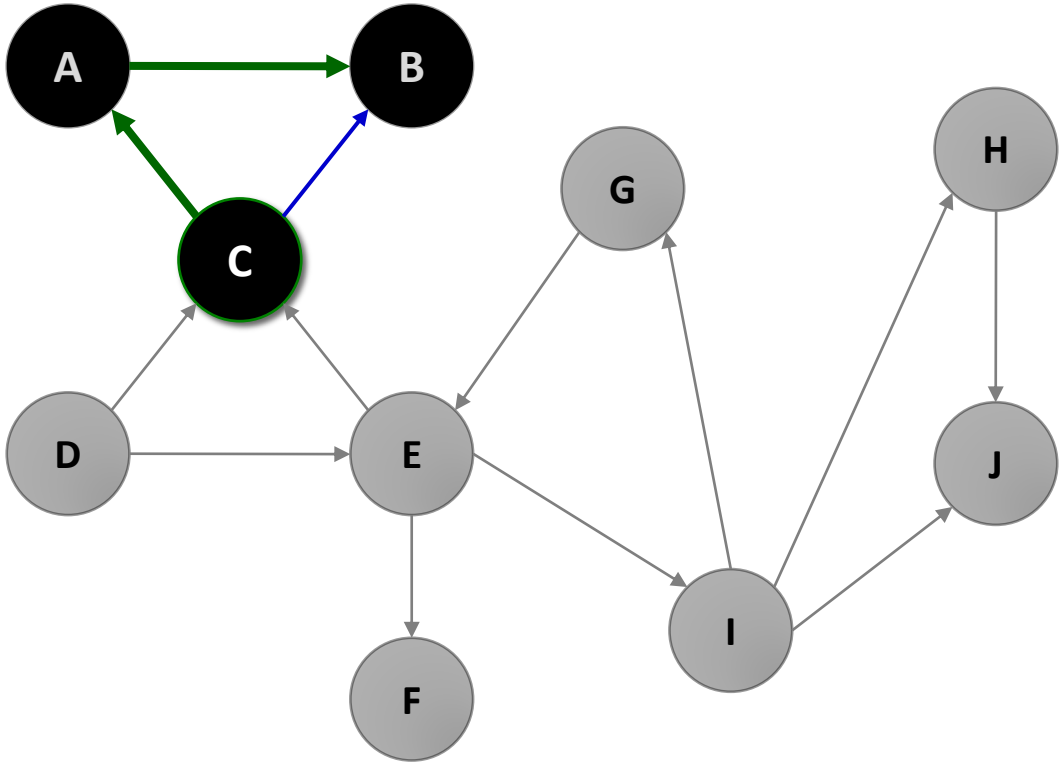
DFS



DFS

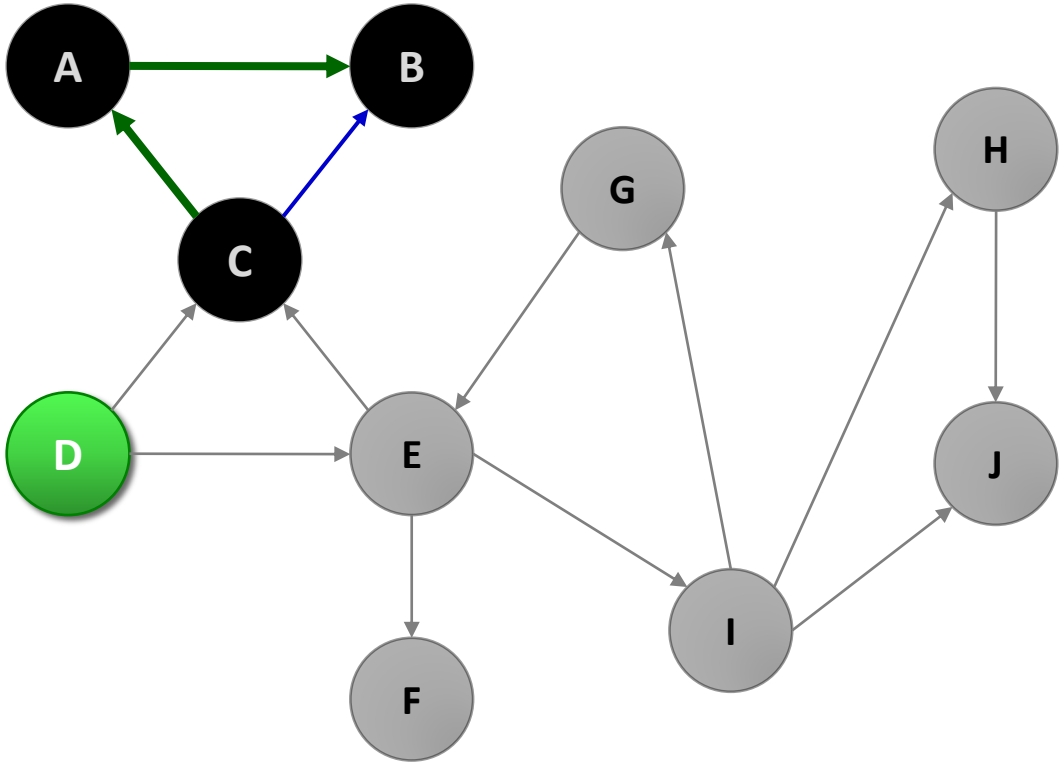


DFS

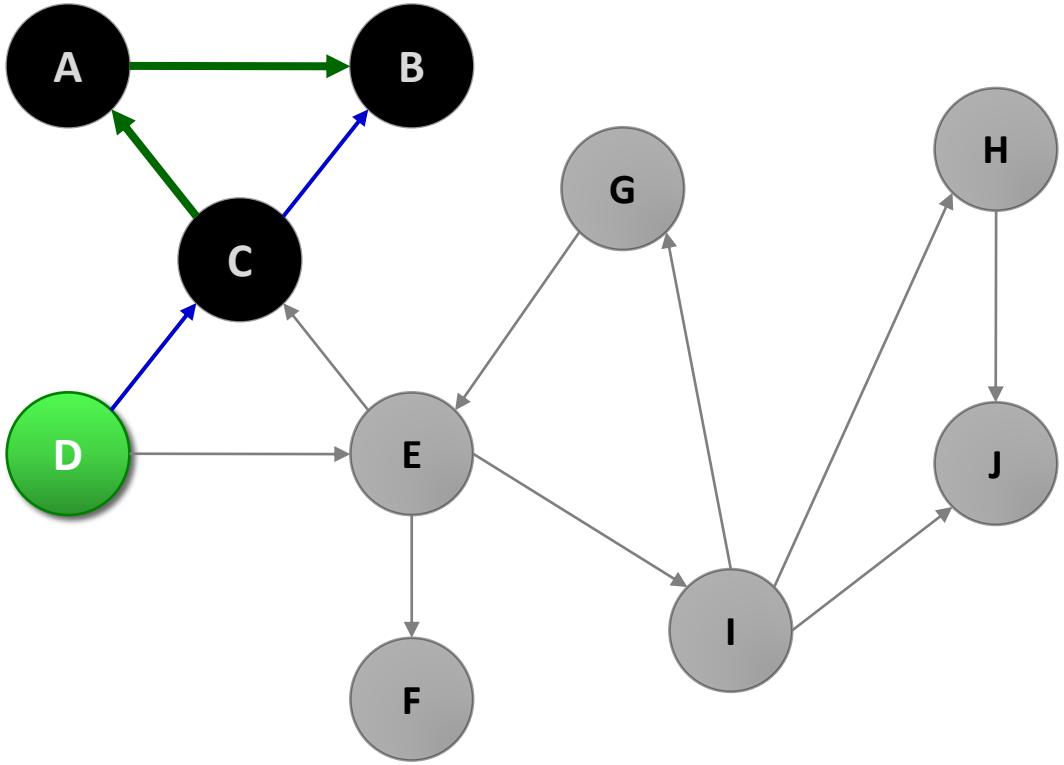


Need to start from a new root among the unmarked vertices

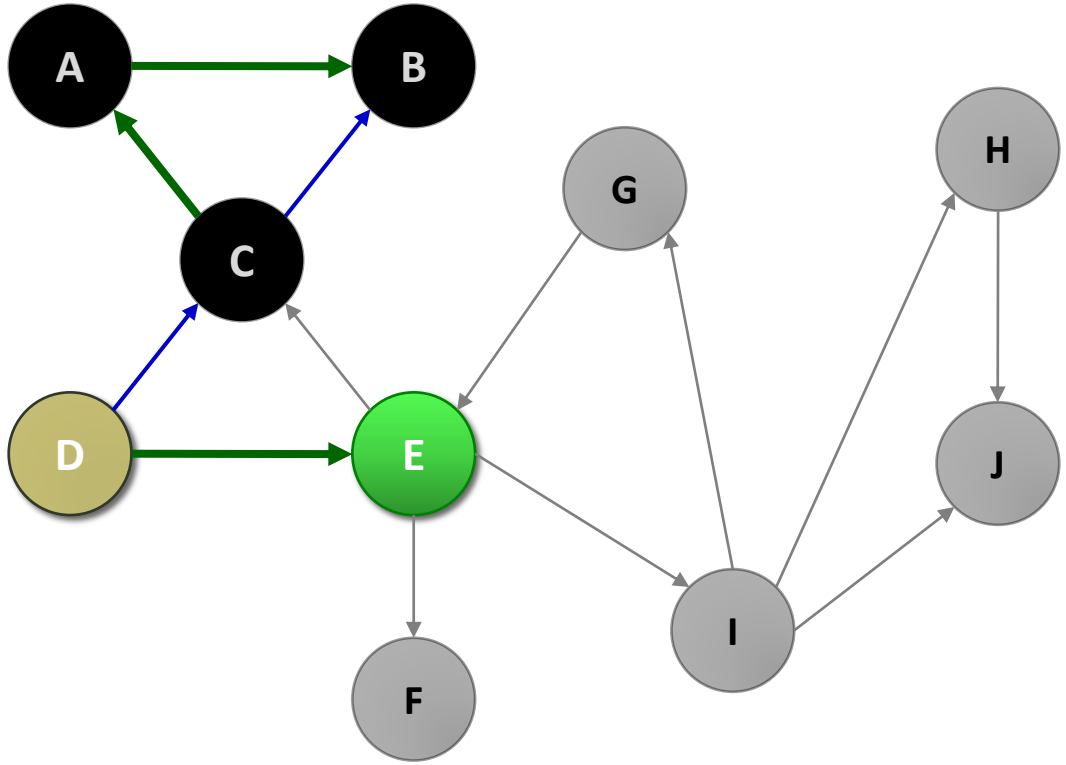
DFS



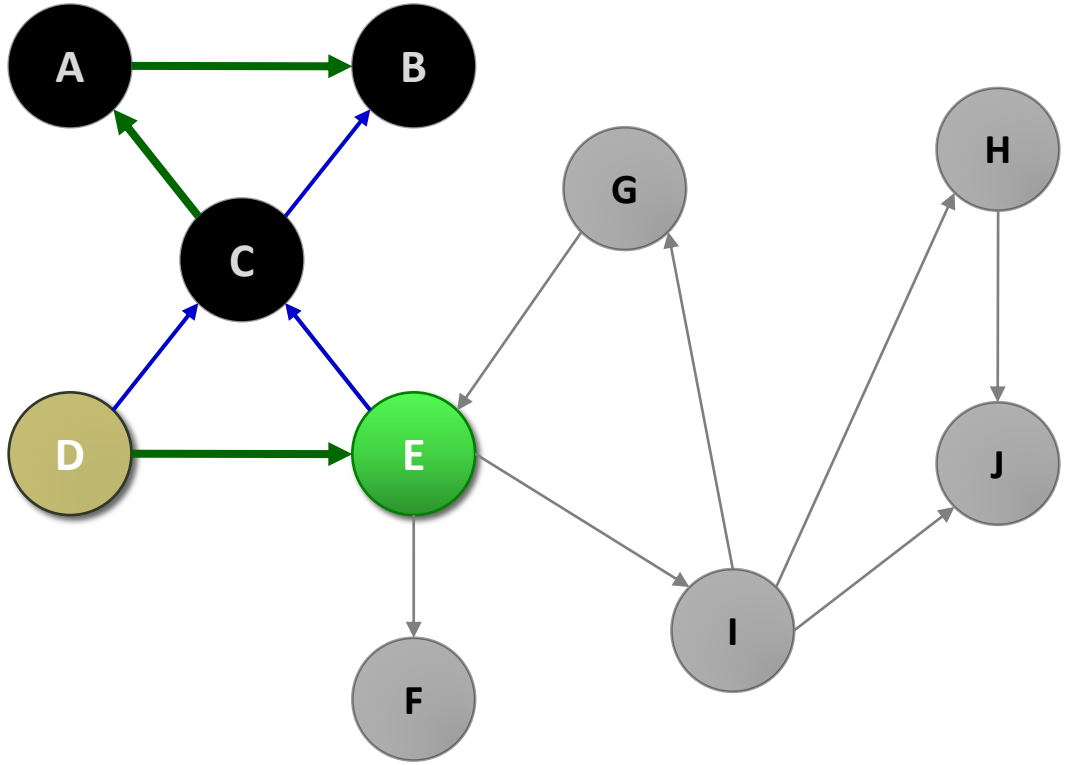
DFS



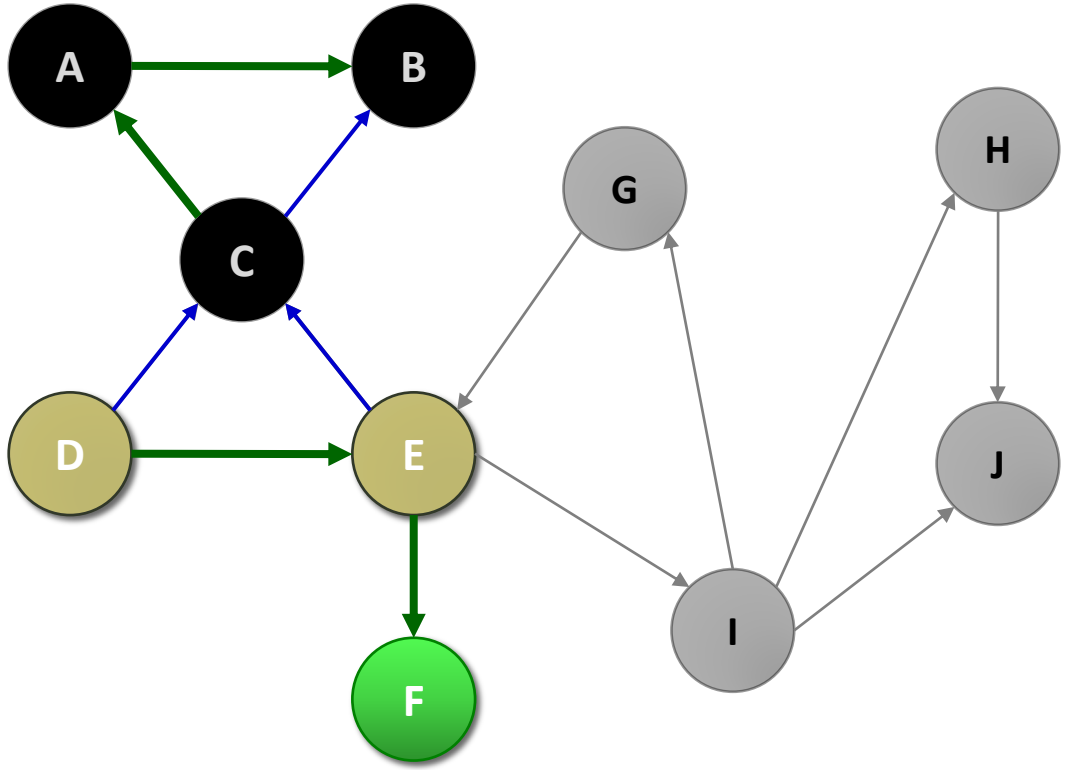
DFS



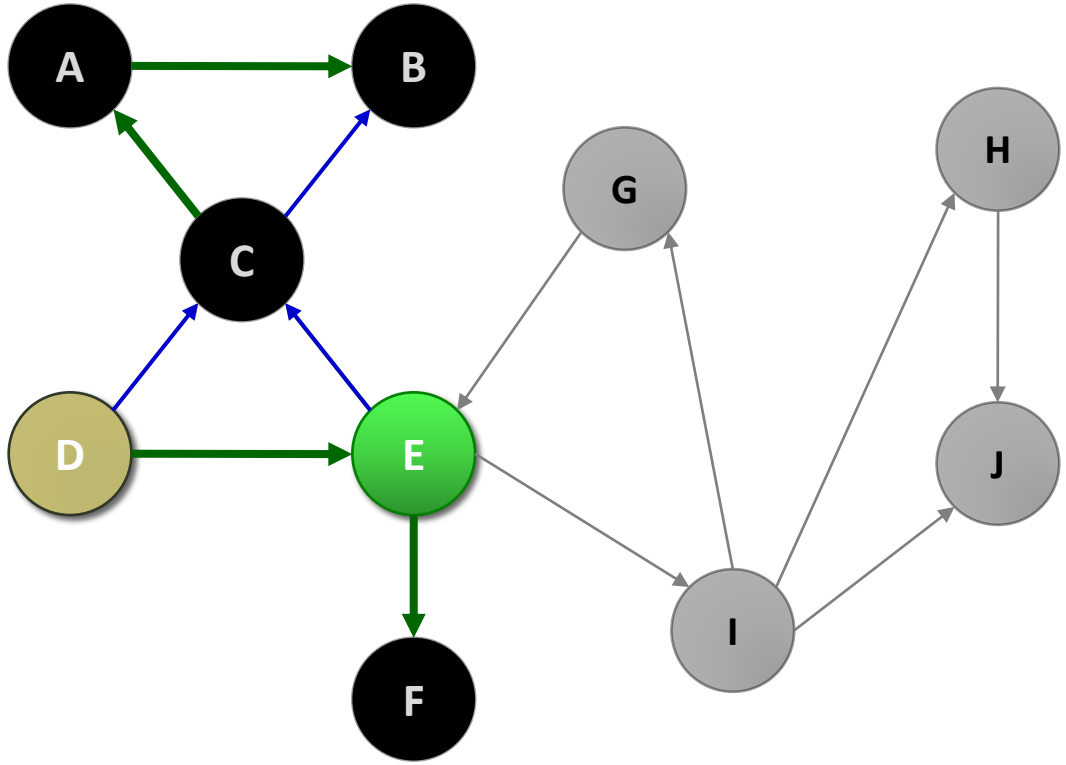
DFS



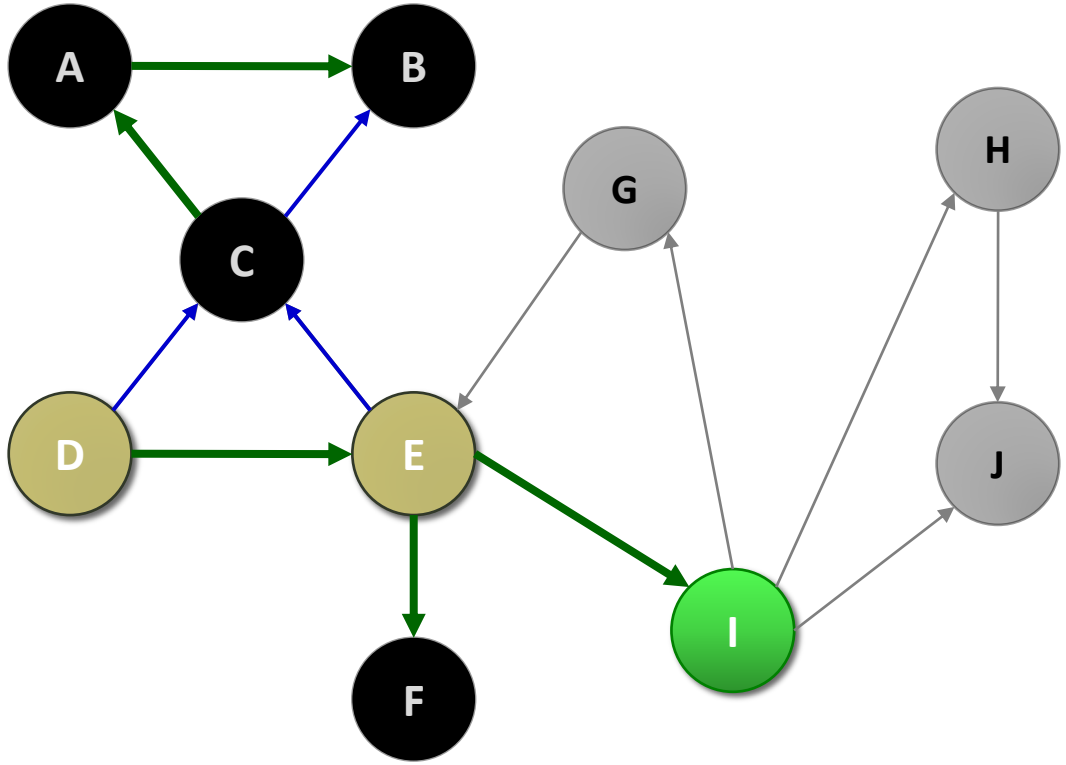
DFS



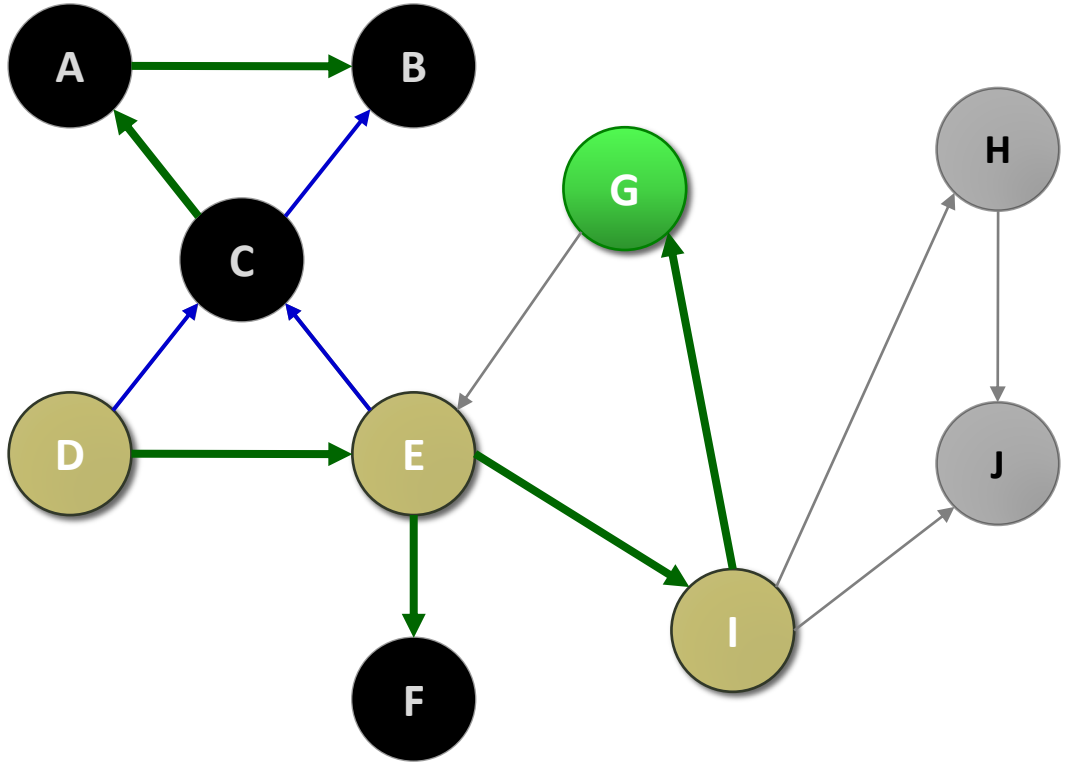
DFS



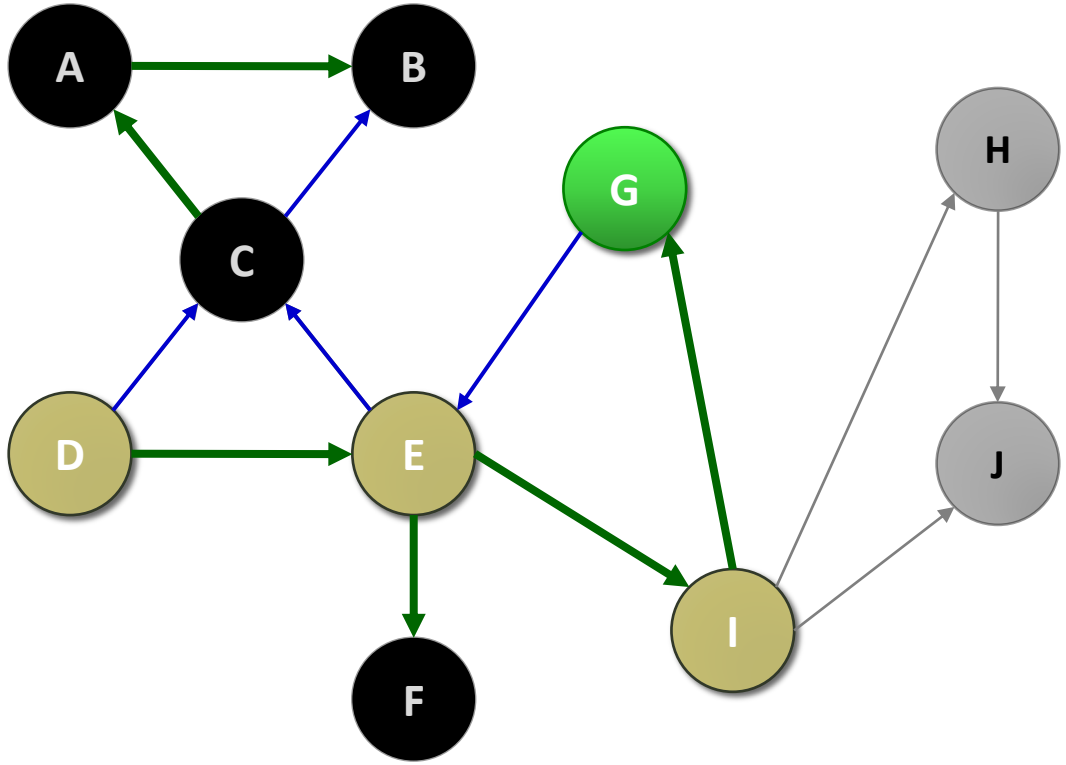
DFS



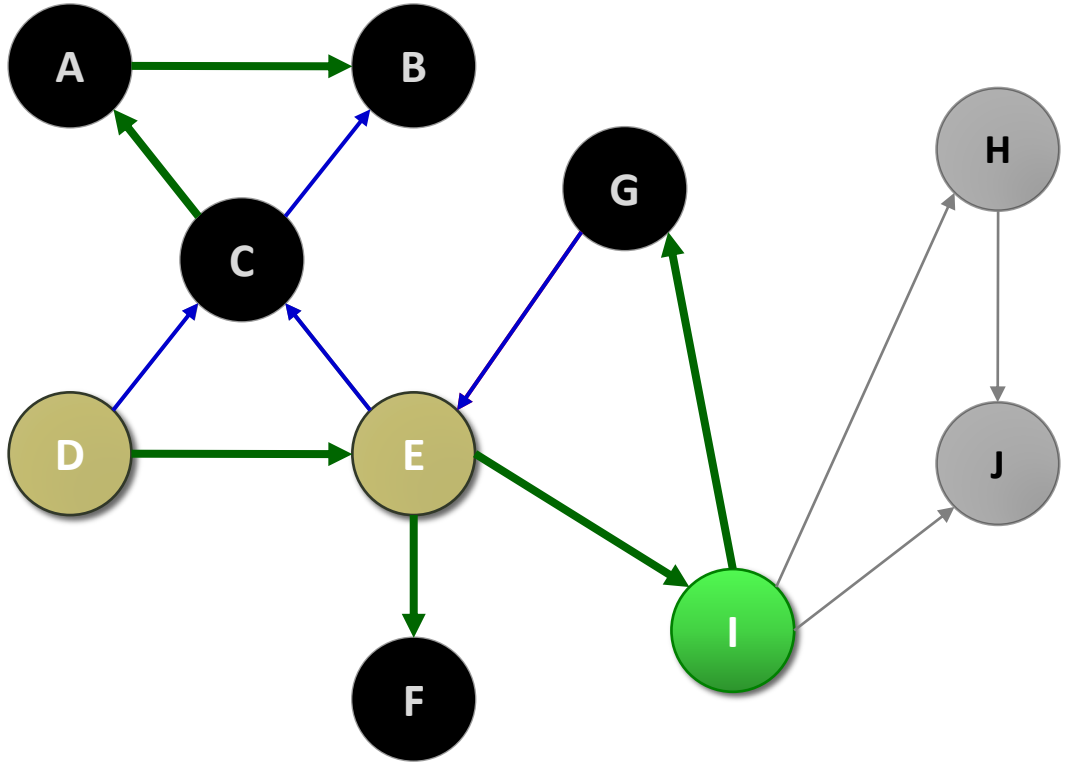
DFS



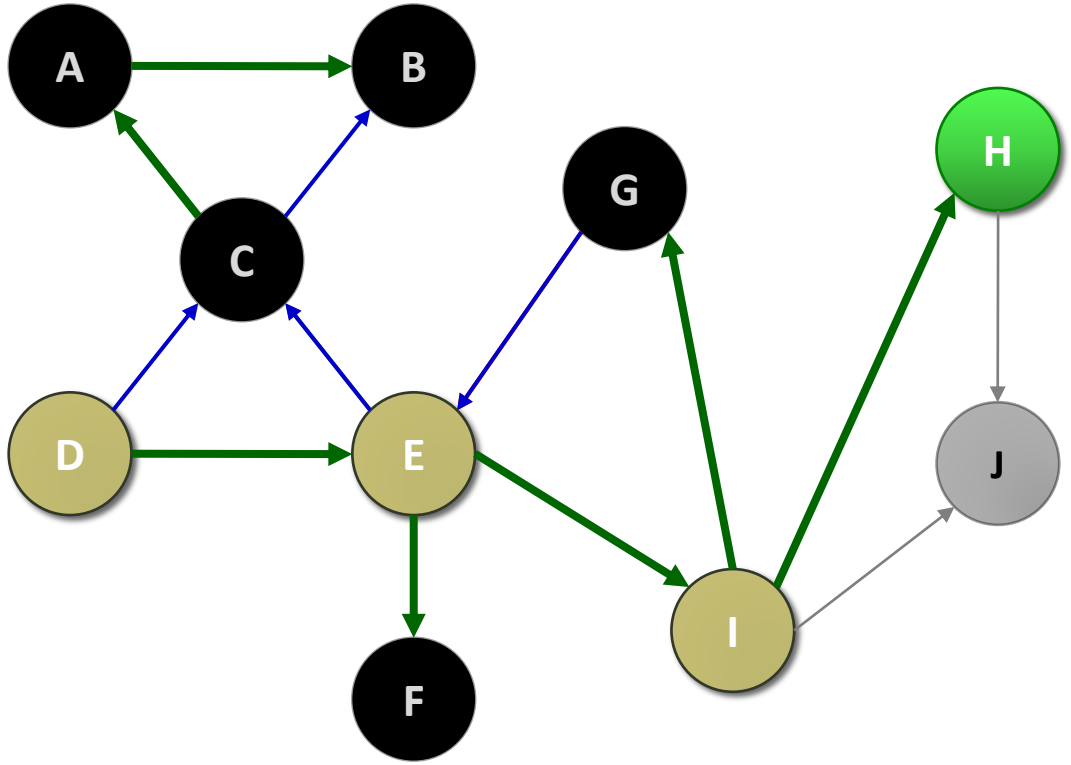
DFS



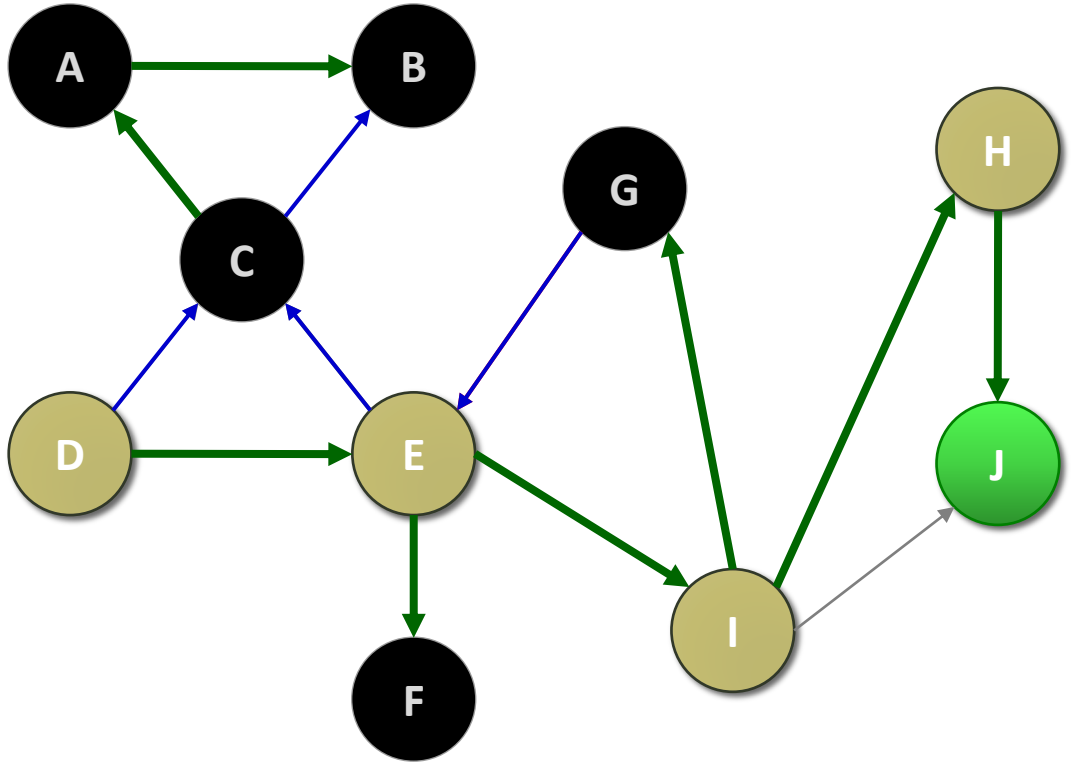
DFS



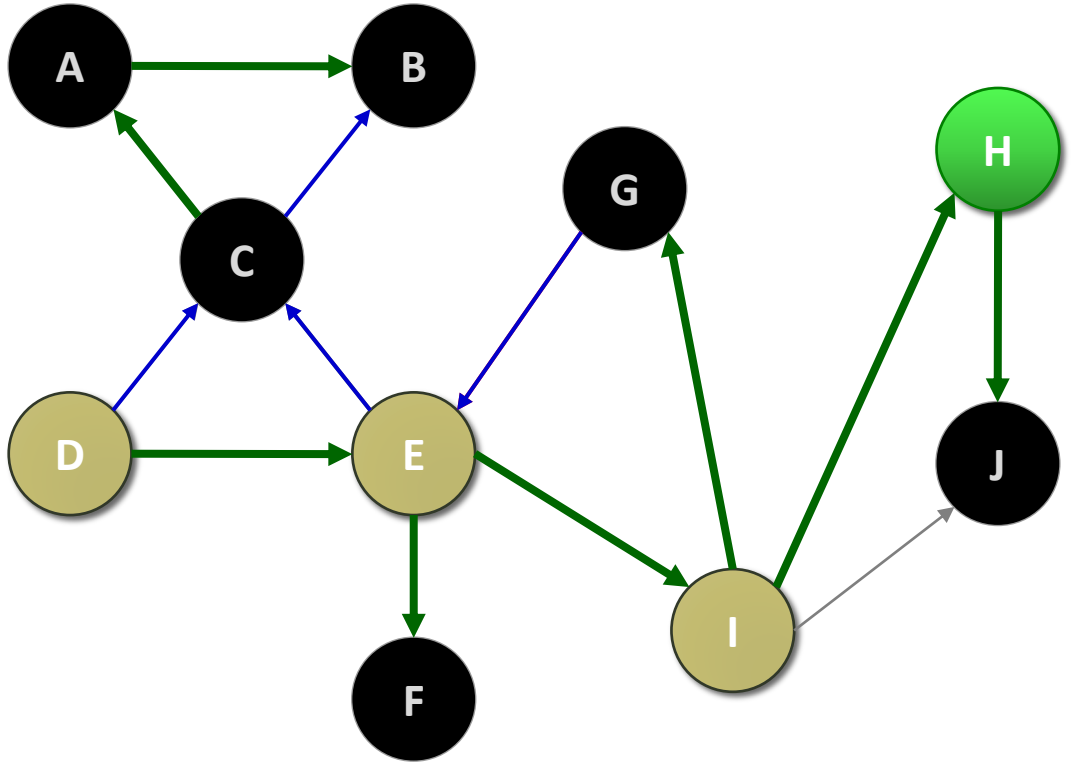
DFS



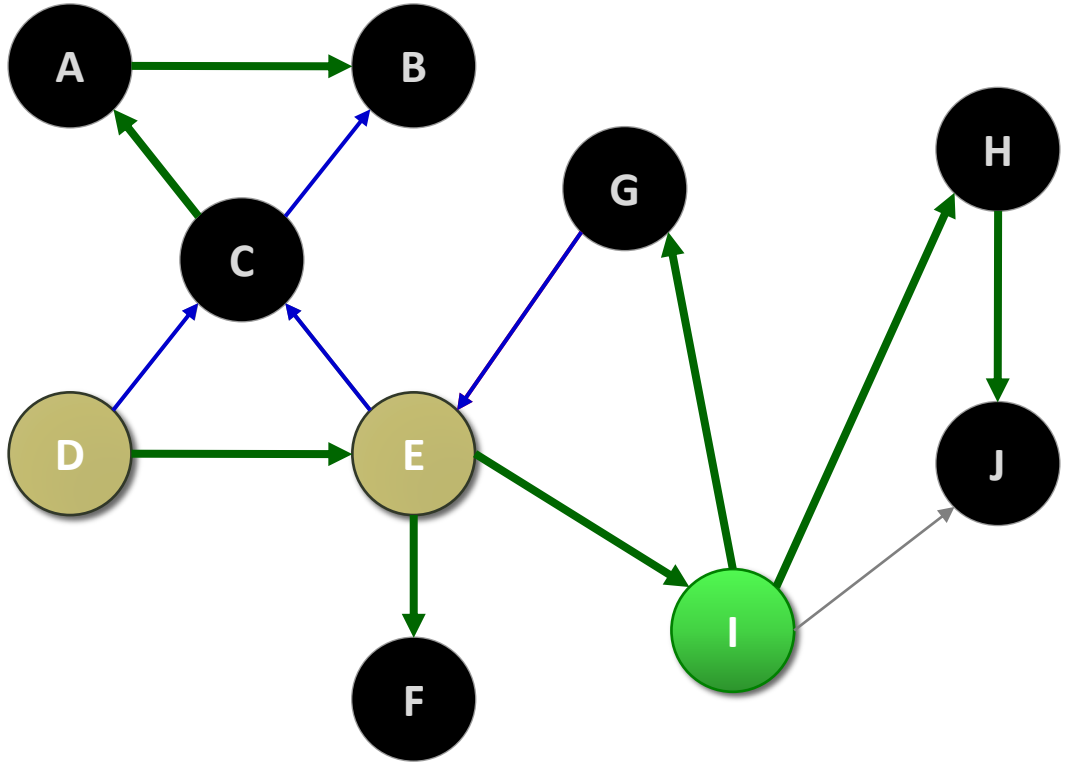
DFS



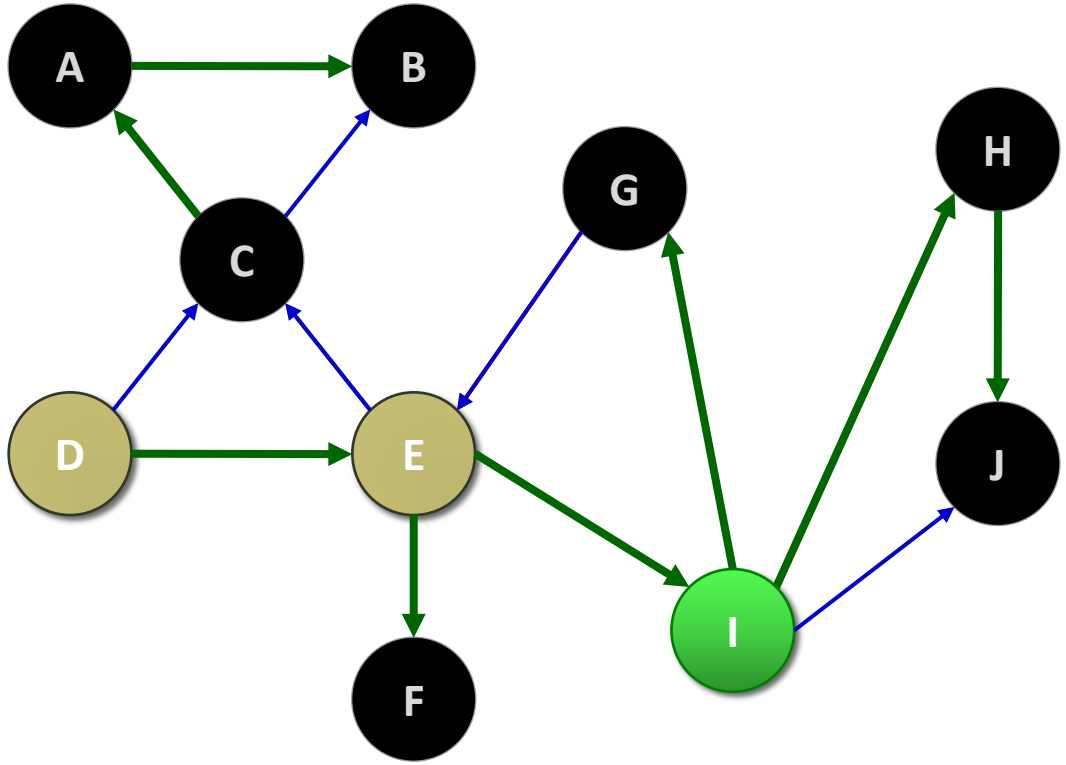
DFS



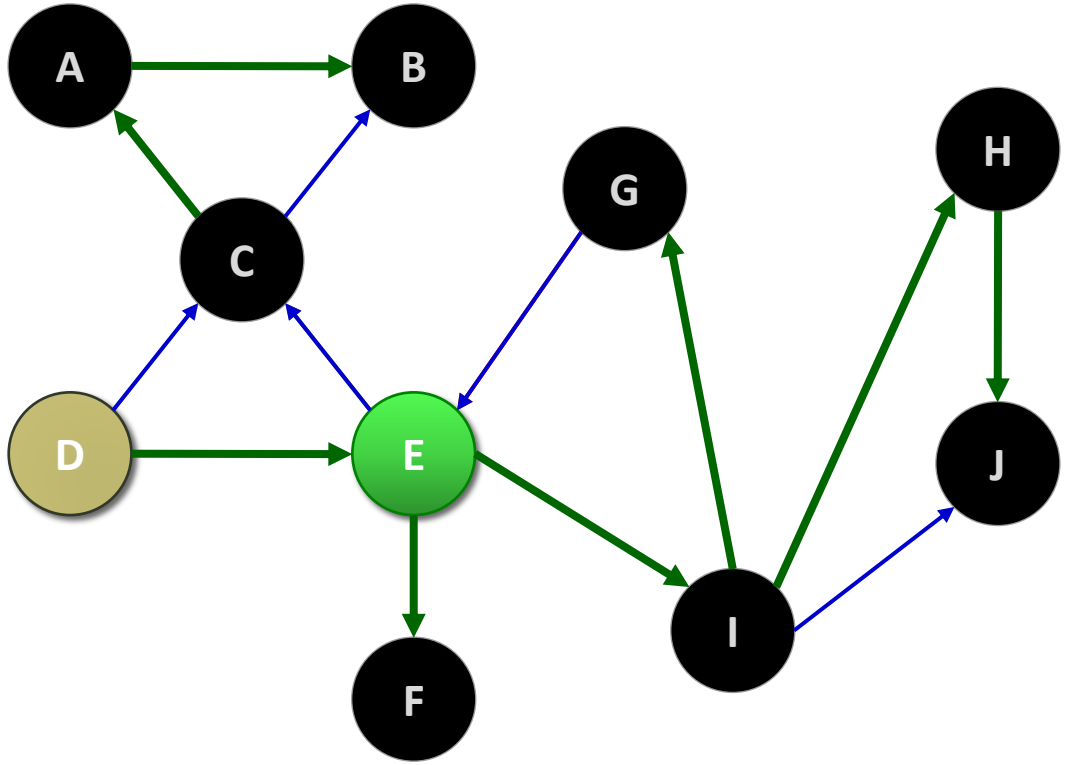
DFS



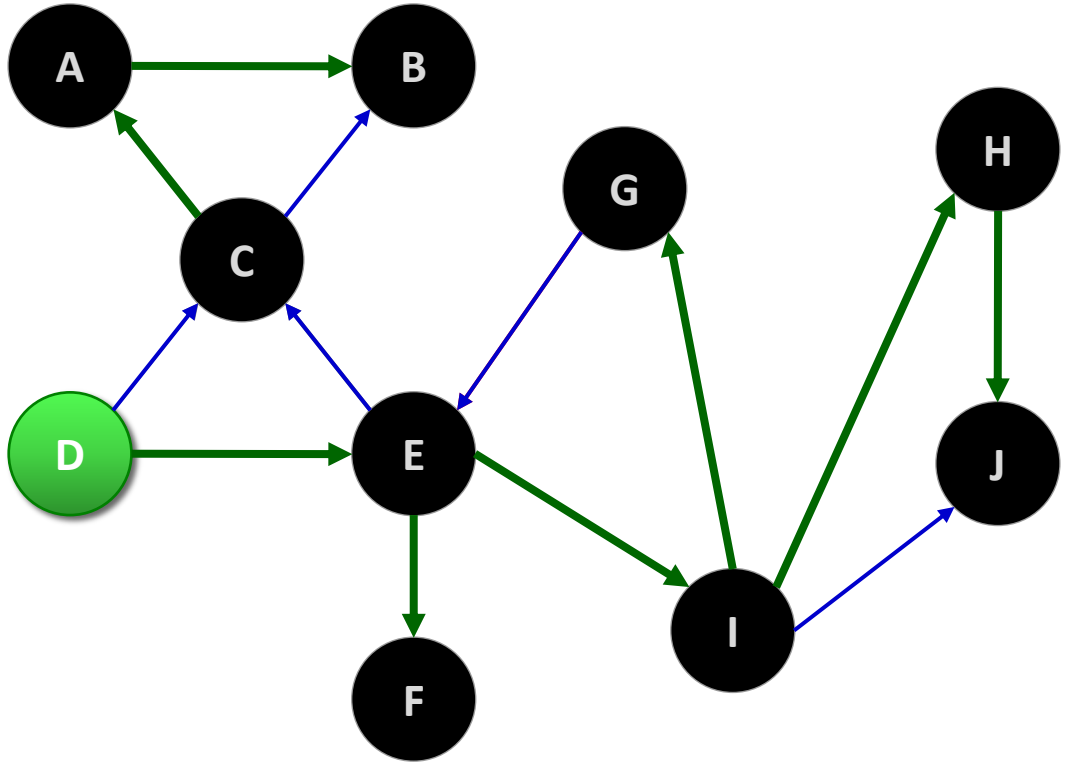
DFS



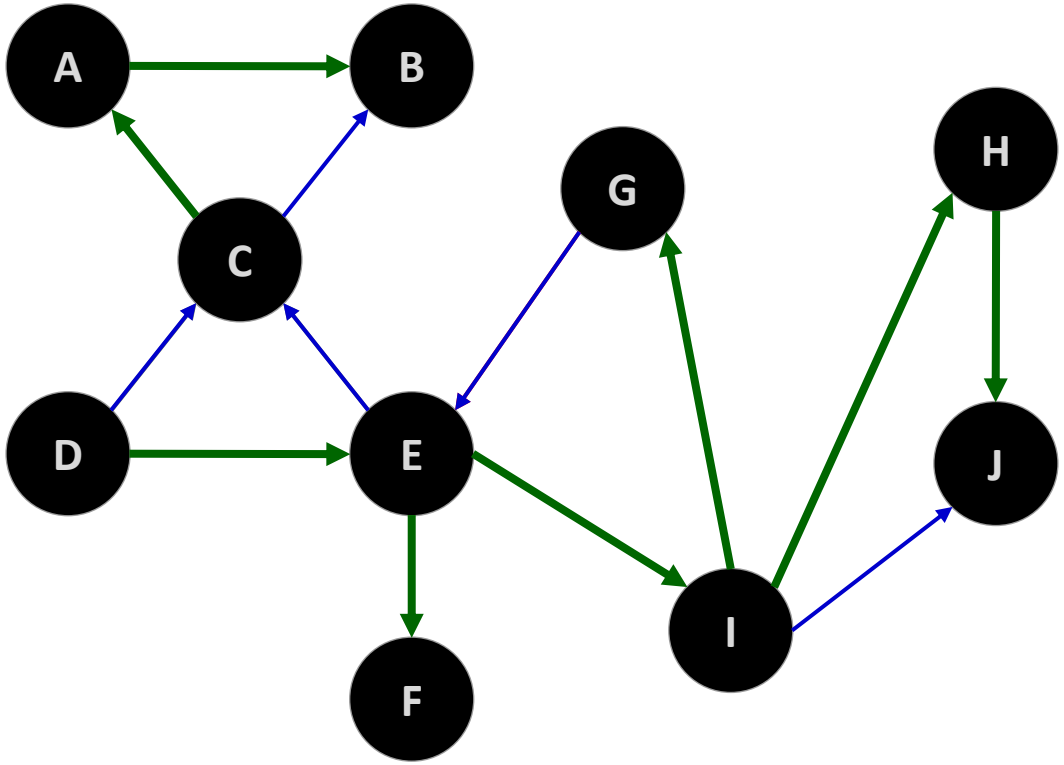
DFS



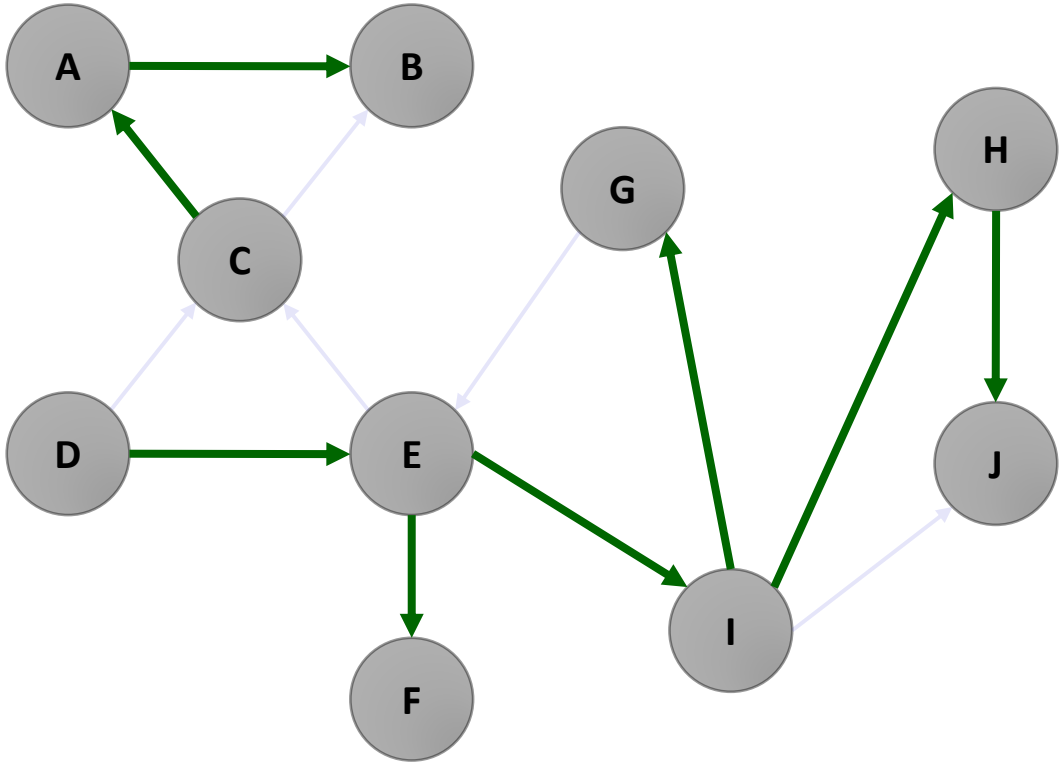
DFS



DFS



DFS – Resulting Forest



- Every tree contains all the vertices reachable from the root
- Resulting forest is different for each initial root!
- DFS(D) produces a tree
- DFS(H) may produce a forest with 3 trees

Exercise 1

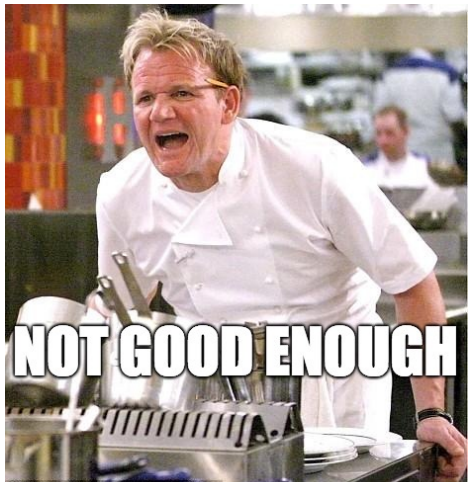
Definition: A **root** is a vertex with a directed path to every vertex in the graph.

Given a directed graph $G = (V, E)$, suggest an algorithm that finds a root in time O(n+m) or informs that no root exists.

Prove the correctness and analyze the running time of your algorithm.

Exercise 1 - Solution

- Intuition: if there is a root, DFS(root) will result in a tree.
- Naïve solution: for every vertex v, run DFS(v). If DFS(v) returns a tree – report v. If none returns a tree – report there is no root.
- Complexity – O(n(n+m)).
- We will see a more efficient solution

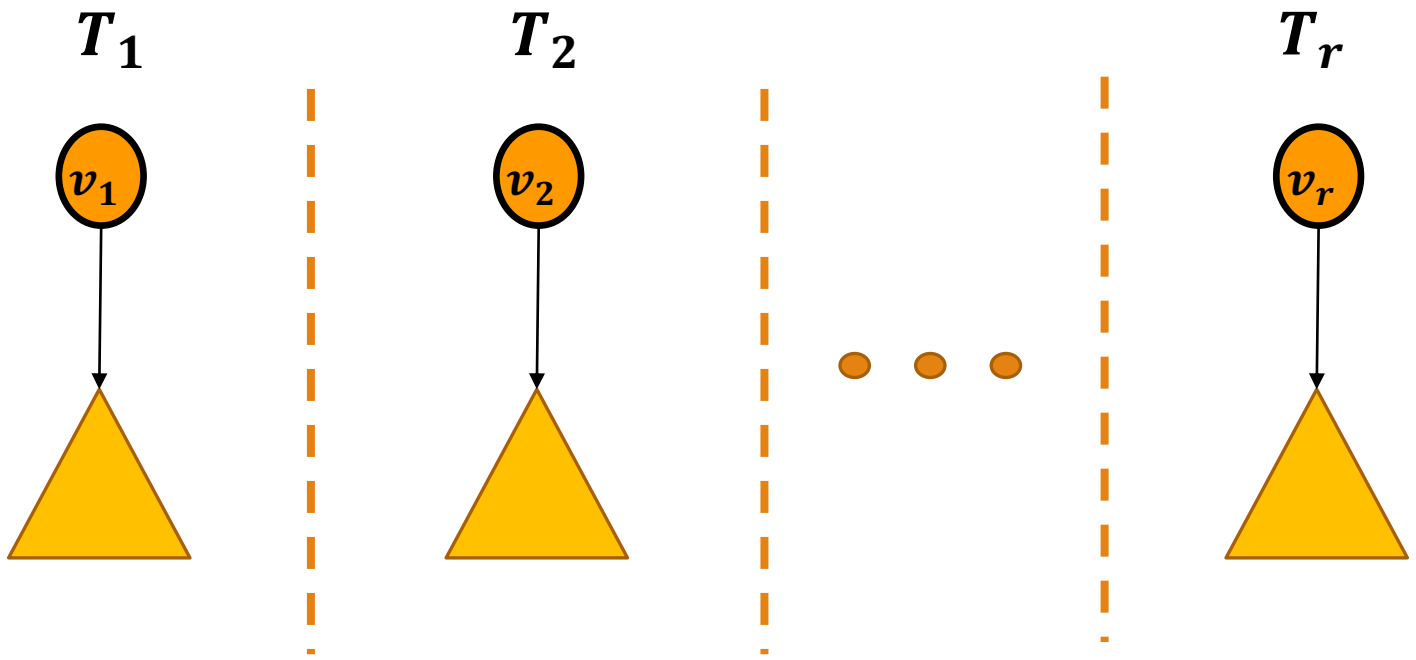


Exercise 1 - Solution

Algorithm:

1. Perform DFS(v) on some random vertex.
2. If we obtain a spanning tree, return v.
3. Else, perform DFS(r), where r is the root of the last tree in the DFS forest obtained in (1).
4. If we obtain a spanning tree, return r.
5. Else, return ”No root in G”.

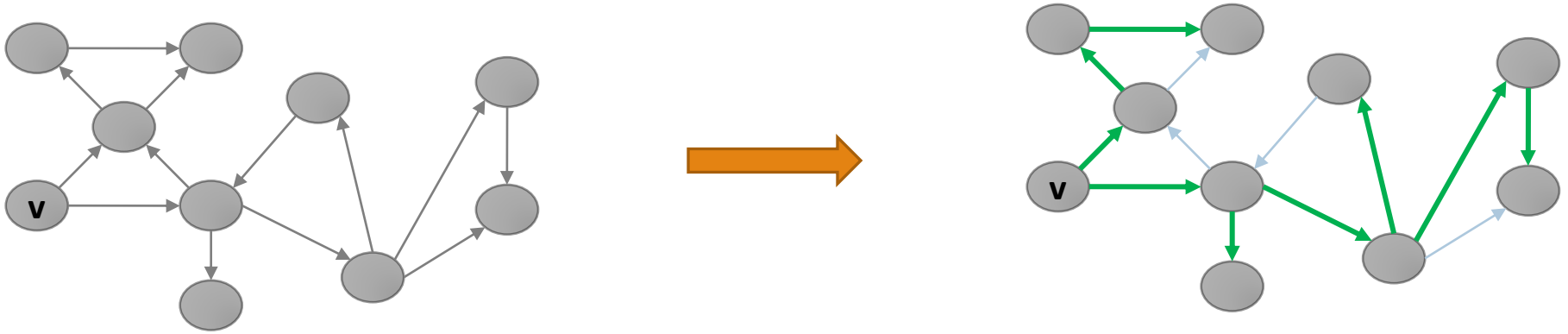
Exercise 1 - Solution



Exercise 1 - Solution

Algorithm:

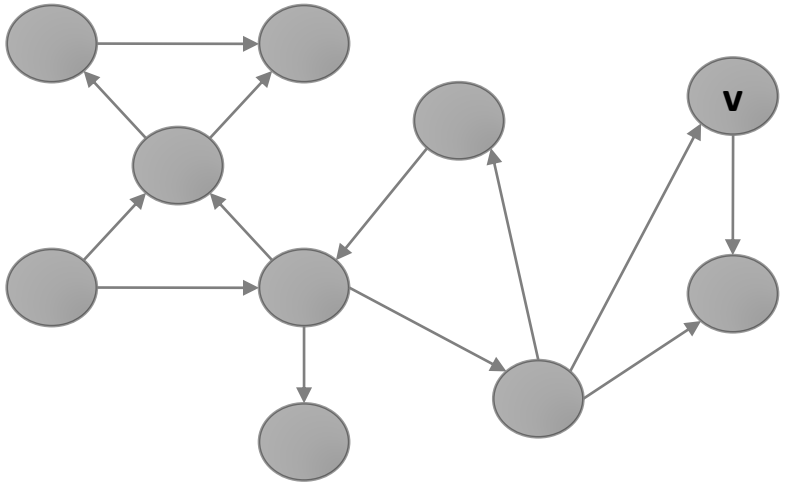
1. **Perform DFS(v) on some random vertex.**
2. **If we obtain a spanning tree, return v.**
3. Else, perform DFS(r), where r is the root of the last tree in the DFS forest obtained in (1).
4. If we obtain a spanning tree, return r.
5. Else, return ”No root in G”.



Exercise 1 - Solution

Algorithm:

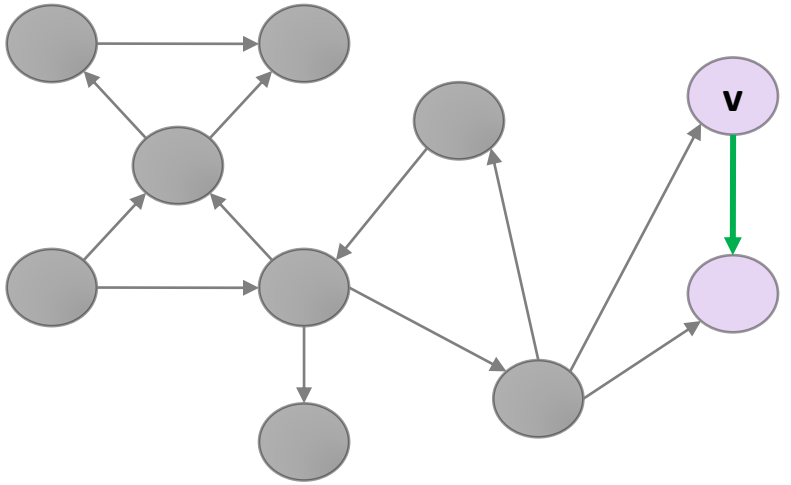
1. **Perform DFS(v) on some random vertex.**
2. If we obtain a spanning tree, return v.
3. Else, perform DFS(r), where r is the root of the last tree in the DFS forest obtained in (1).
4. If we obtain a spanning tree, return r.
5. Else, return ”No root in G”.



Exercise 1 - Solution

Algorithm:

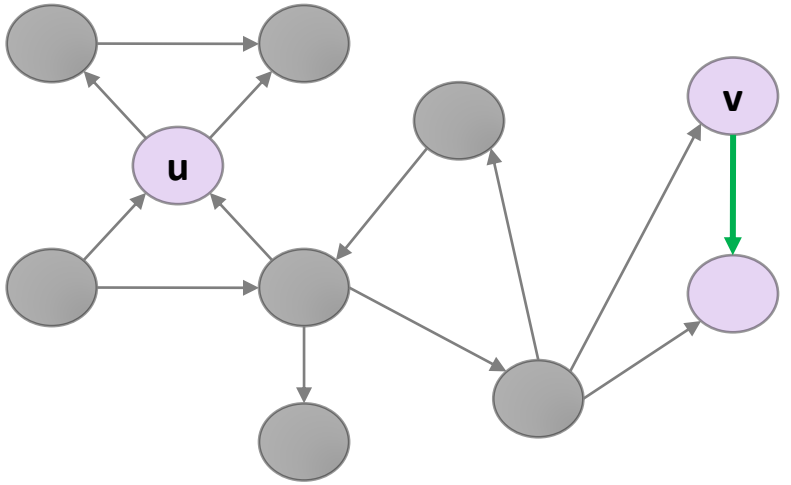
1. **Perform DFS(v) on some random vertex.**
2. If we obtain a spanning tree, return v.
3. Else, perform DFS(r), where r is the root of the last tree in the DFS forest obtained in (1).
4. If we obtain a spanning tree, return r.
5. Else, return ”No root in G”.



Exercise 1 - Solution

Algorithm:

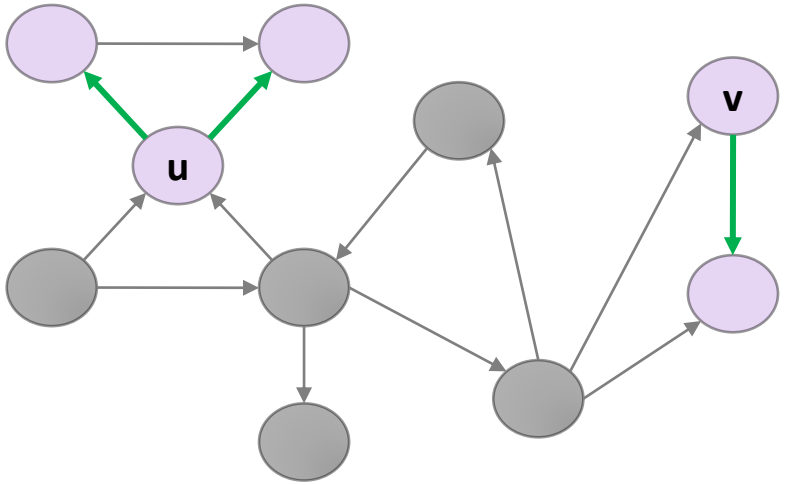
1. **Perform DFS(v) on some random vertex.**
2. If we obtain a spanning tree, return v.
3. Else, perform DFS(r), where r is the root of the last tree in the DFS forest obtained in (1).
4. If we obtain a spanning tree, return r.
5. Else, return ”No root in G”.



Exercise 1 - Solution

Algorithm:

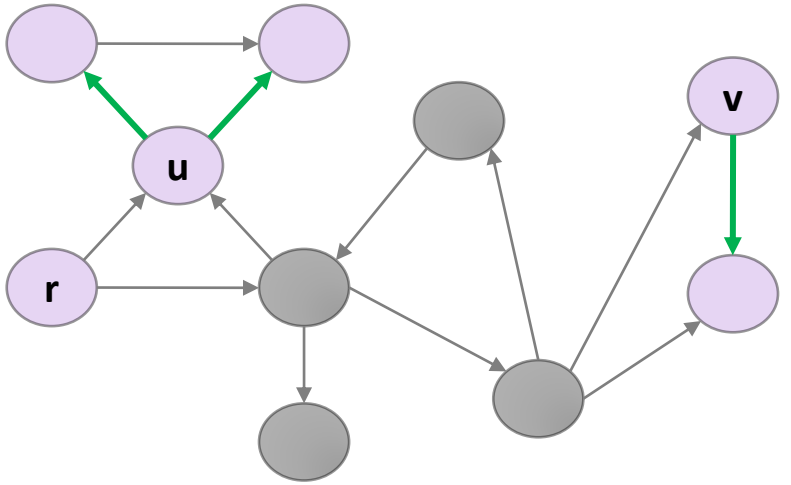
1. **Perform DFS(v) on some random vertex.**
2. If we obtain a spanning tree, return v.
3. Else, perform DFS(r), where r is the root of the last tree in the DFS forest obtained in (1).
4. If we obtain a spanning tree, return r.
5. Else, return ”No root in G”.



Exercise 1 - Solution

Algorithm:

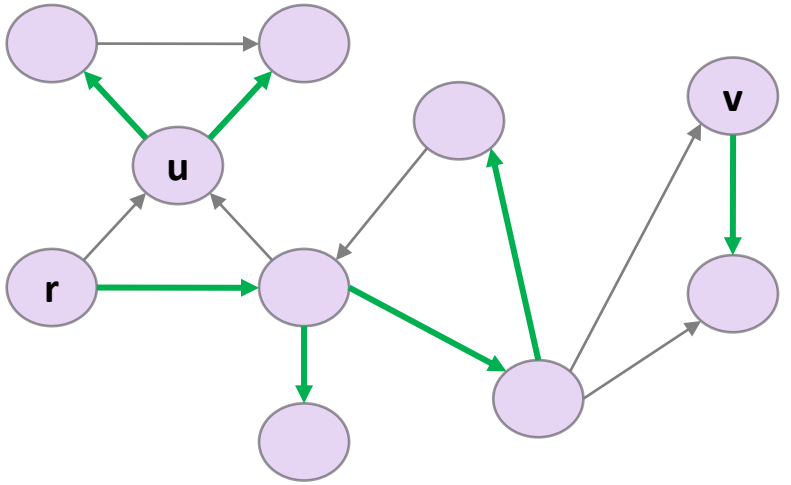
1. **Perform DFS(v) on some random vertex.**
2. If we obtain a spanning tree, return v.
3. Else, perform DFS(r), where r is the root of the last tree in the DFS forest obtained in (1).
4. If we obtain a spanning tree, return r.
5. Else, return ”No root in G”.



Exercise 1 - Solution

Algorithm:

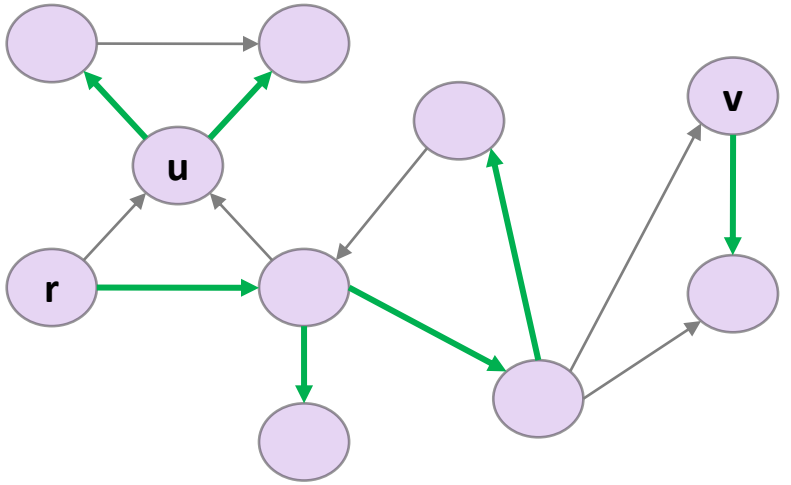
1. **Perform DFS(v) on some random vertex.**
2. If we obtain a spanning tree, return v.
3. Else, perform DFS(r), where r is the root of the last tree in the DFS forest obtained in (1).
4. If we obtain a spanning tree, return r.
5. Else, return ”No root in G”.



Exercise 1 - Solution

Algorithm:

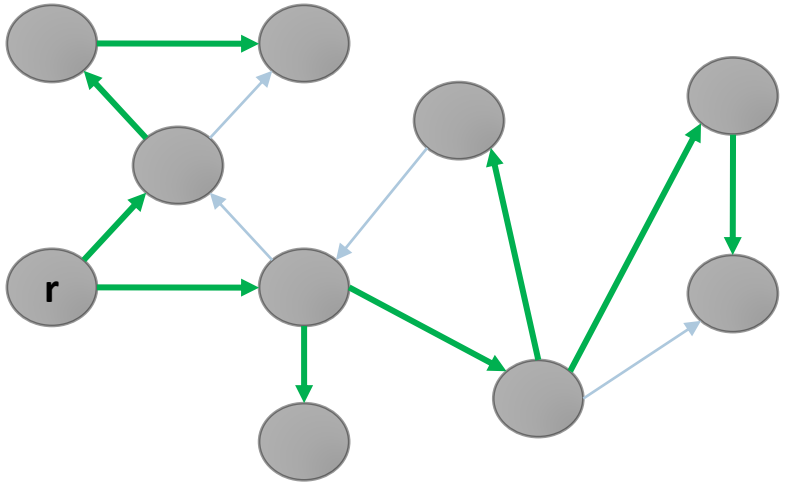
1. **Perform DFS(v) on some random vertex.**
2. If we obtain a spanning tree, return v.
3. **Else, perform DFS(r), where r is the root of the last tree in the DFS forest obtained in (1).**
4. If we obtain a spanning tree, return r.
5. Else, return ”No root in G”.



Exercise 1 - Solution

Algorithm:

1. **Perform DFS(v) on some random vertex.**
2. If we obtain a spanning tree, return v.
3. **Else, perform DFS(r), where r is the root of the last tree in the DFS forest obtained in (1).**
4. If we obtain a spanning tree, return r.
5. Else, return ”No root in G”.



Exercise 1 - Solution

Correctness:

If the algorithm return some vertex as the root, then it is indeed a root, since we obtain a spanning tree when performing DFS on that vertex.

We need to show that if the algorithm returns that a root does not exist, that is, if r, which is the only candidate root tested by the algorithm, is not a root, then no other vertex is a root.

Exercise 1 - Solution

Correctness (continued):

Assume towards contradiction that our algorithm outputs ”No root exists”, while some vertex $w \neq r$ is a root

Consider the following cases:

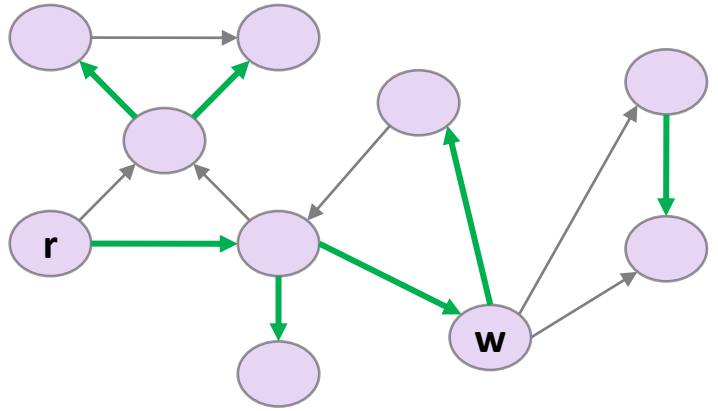
- w belongs to the tree that is rooted by r.
- w doesn’t belong to the tree that is rooted by r.

Exercise 1 - Solution

Correctness (continued):

- w belongs to the tree that is rooted by r:

There is a path from r to w and there is a path from w to every other vertex in the graph. Thus, there must be a path from r to every other vertex in the graph, through w, meaning that r is a root, which is a contradiction since r is not a root.

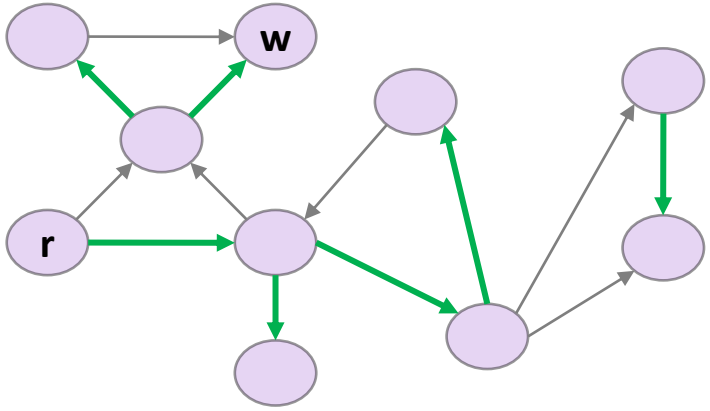


Exercise 1 - Solution

Correctness (continued):

- w doesn’t belong to the tree that is rooted by r:

Since r is the root of the last tree in the DFS forest, w was marked before r. If w is a root, then r was supposed to be discovered and be part of the tree to w belongs, which is a contradiction.



Exercise 1 - Solution

Complexity:

We perform 2 DFS in the worst case. Thus, the running time is $2 \cdot O(m+n) = O(m+n)$.

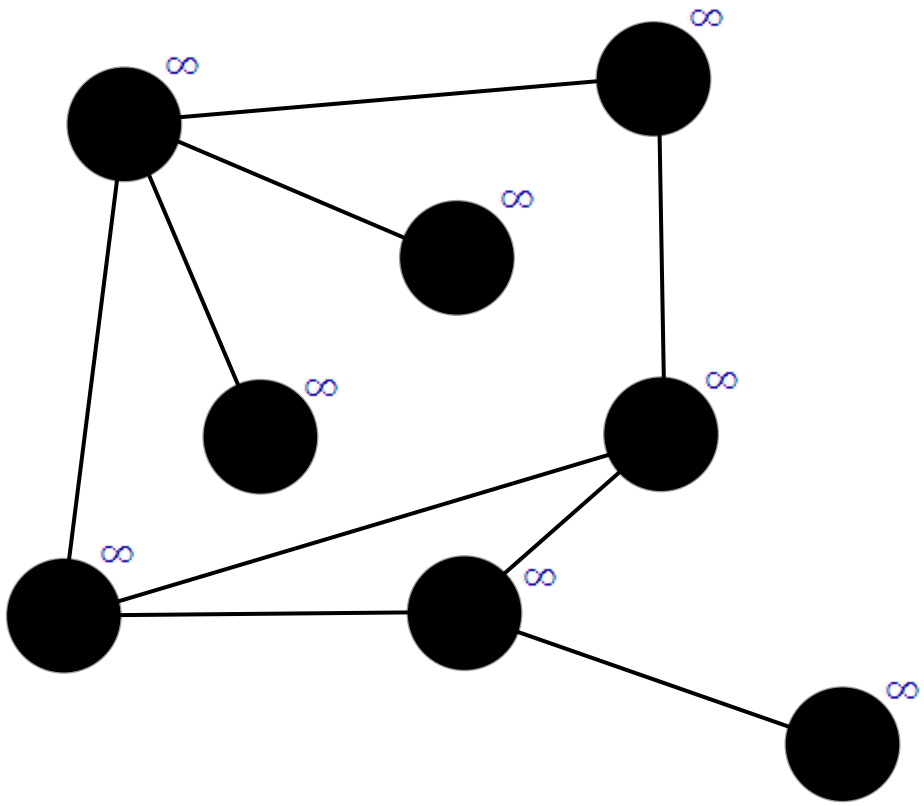
BFS

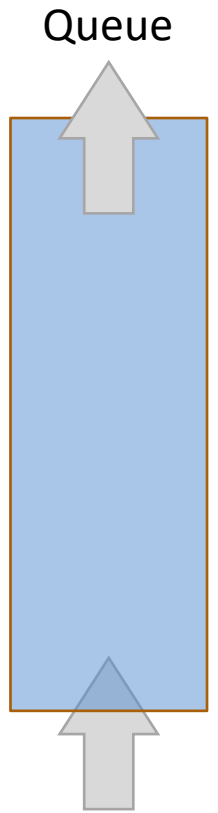
Algorithm 3 BFS(G,v)

$Q \leftarrow \emptyset, T \leftarrow \emptyset$
Enqueue(Q,v) and mark v
While Q is not empty:
    $u \leftarrow Dequeue(Q)$
    For each w adjacent to u:
        if w is not marked then
            add (u, w) to T
            mark w
            Enqueue(Q,w)

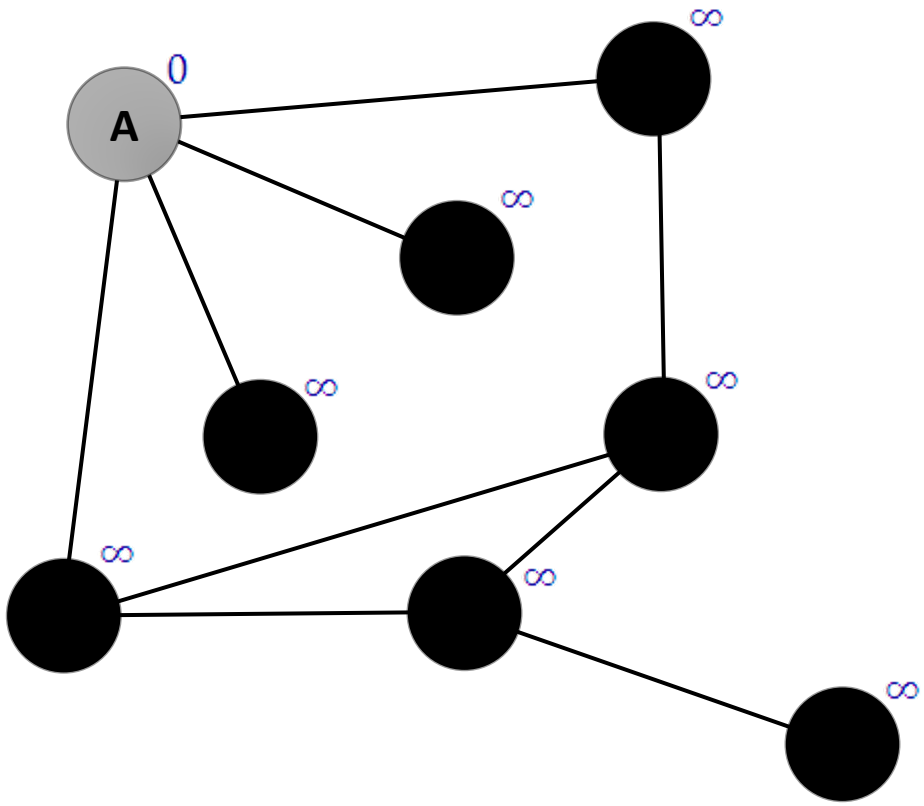
Complexity? O(n+m)

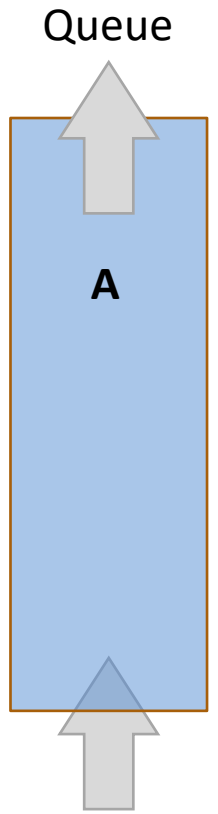
BFS



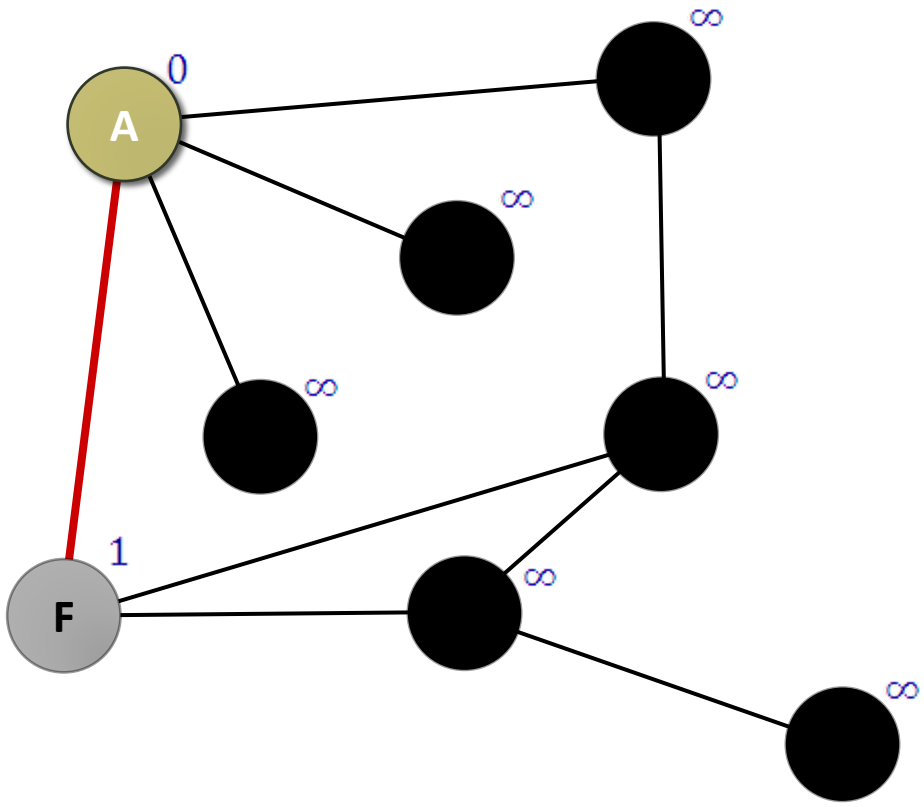


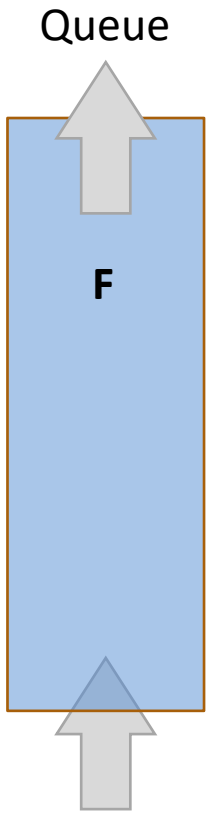
BFS



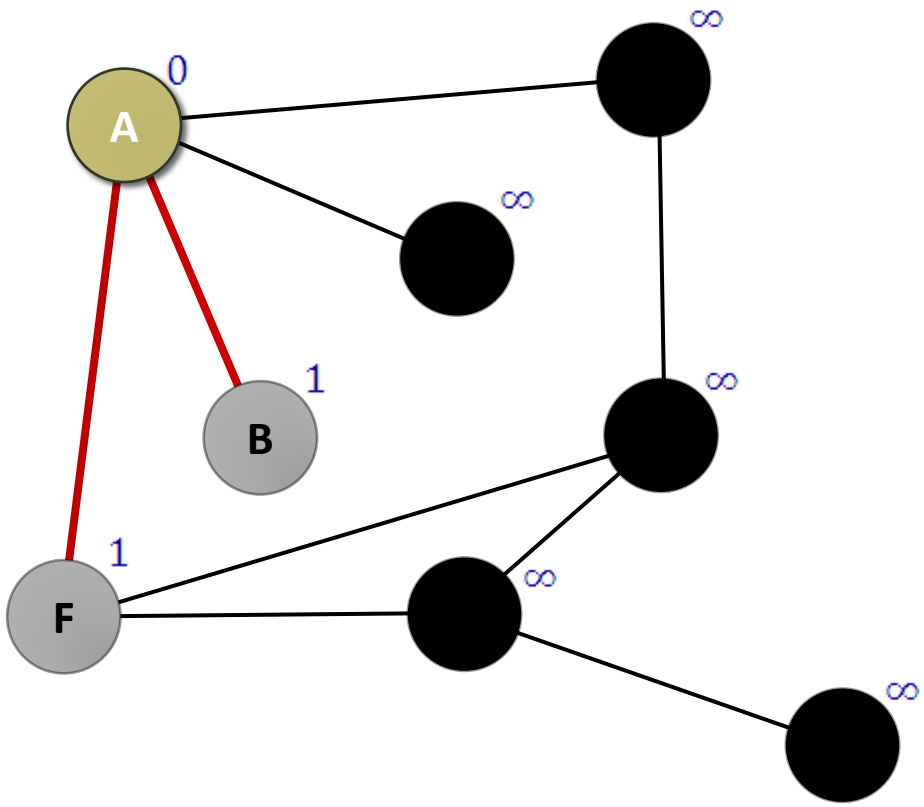


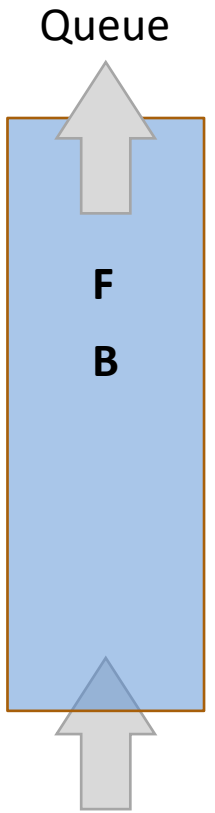
BFS



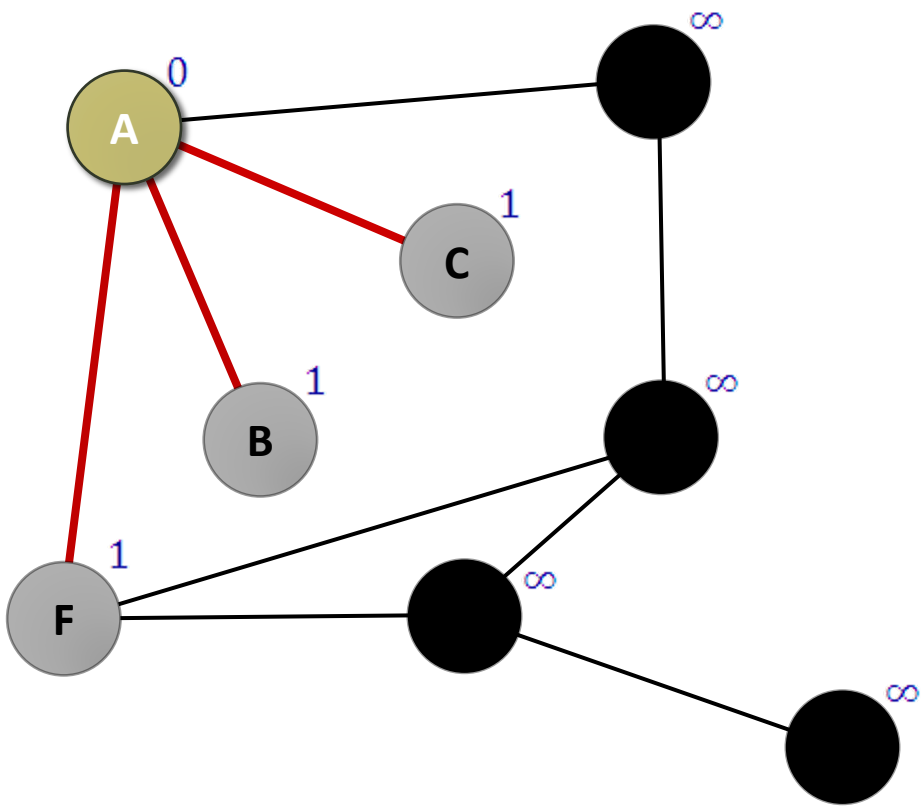


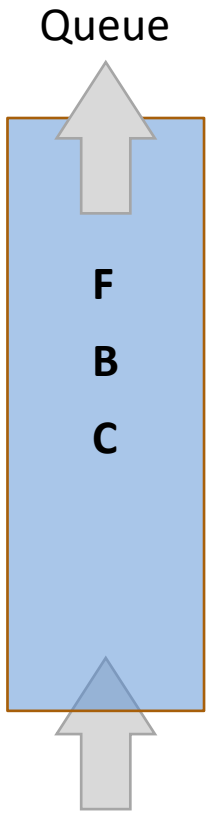
BFS



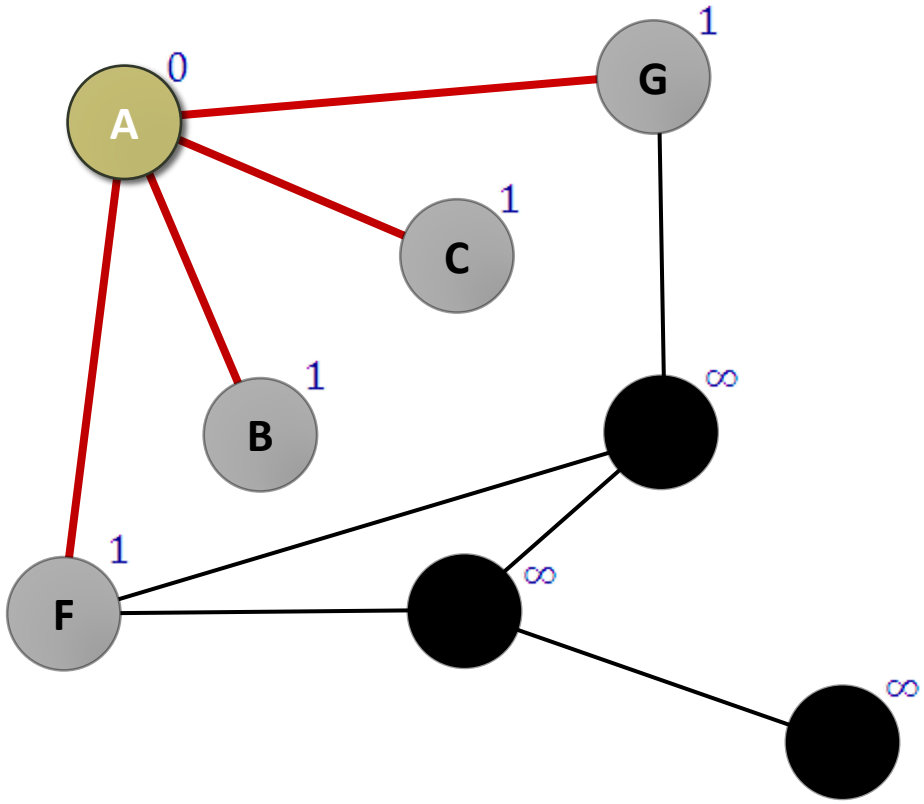


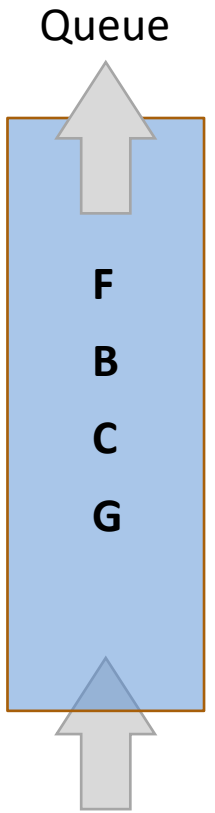
BFS



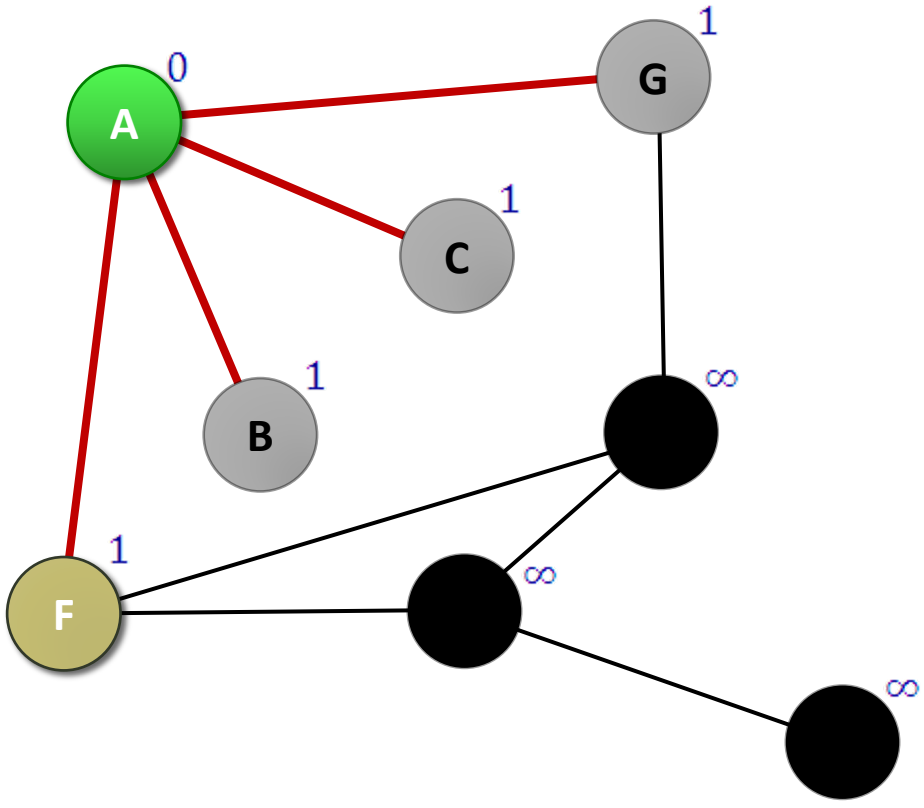


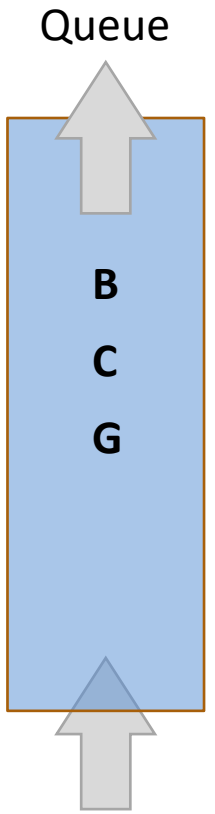
BFS



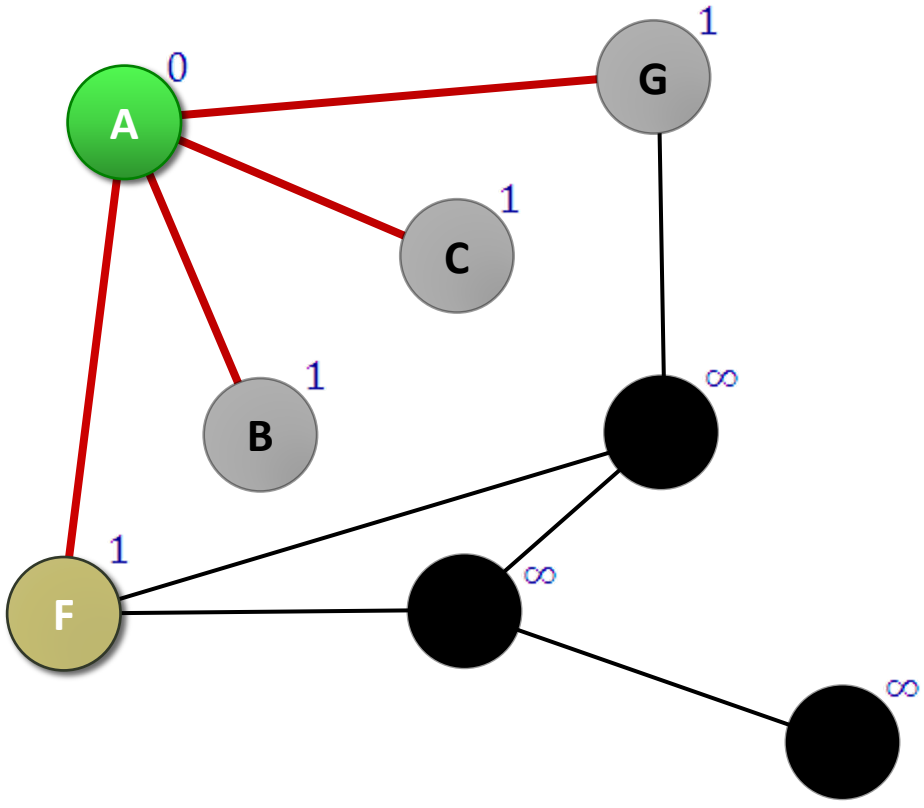


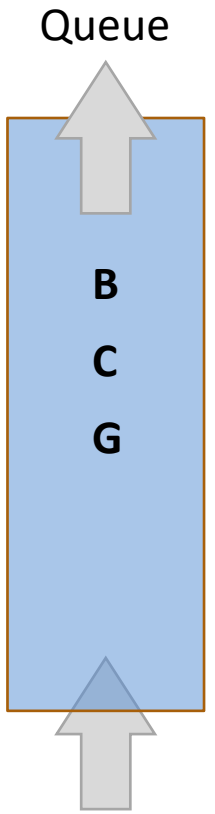
BFS



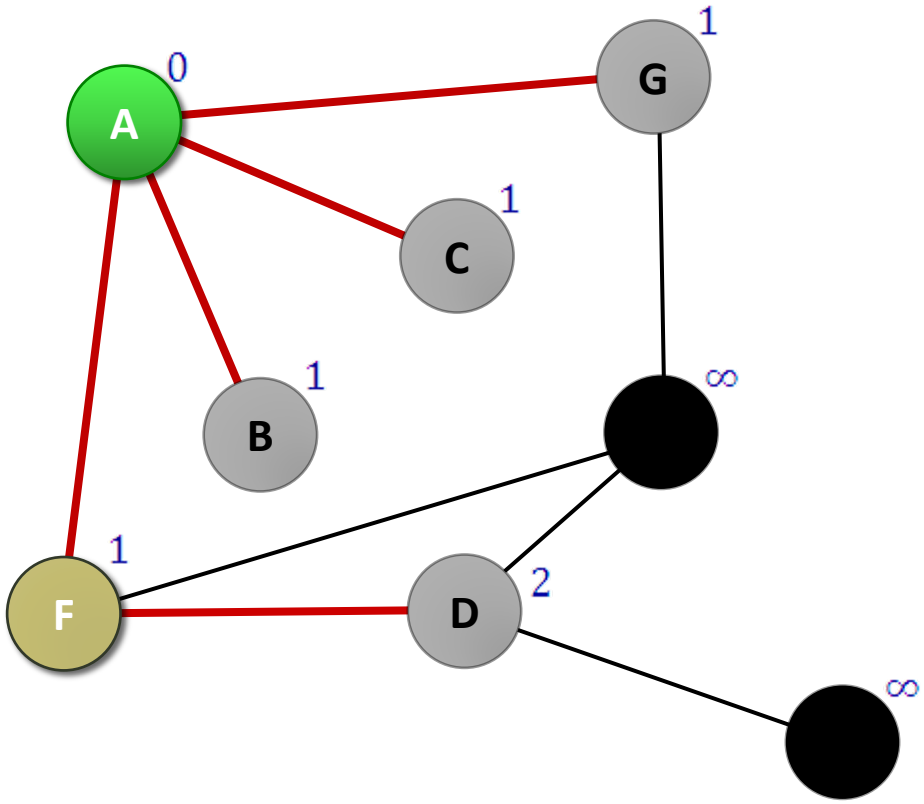


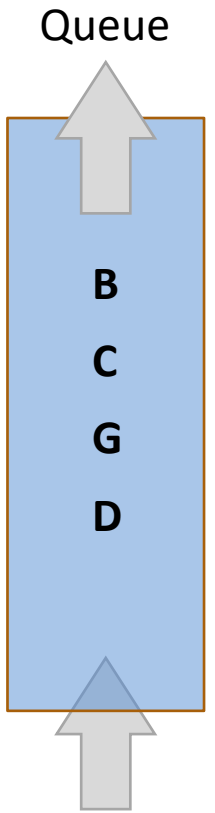
BFS



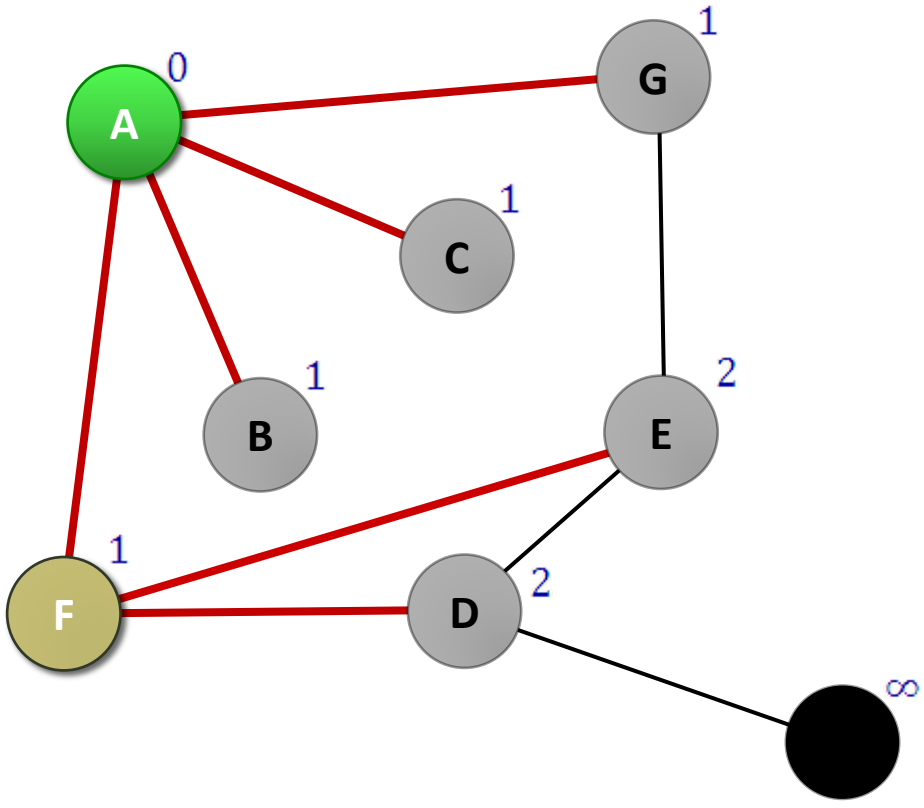


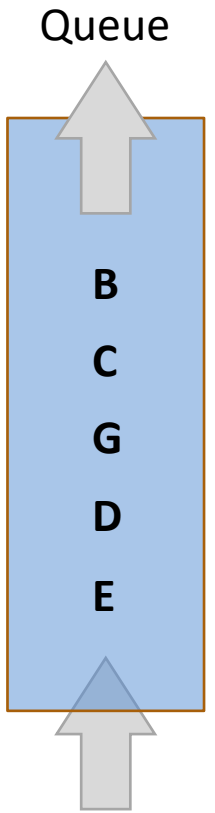
BFS



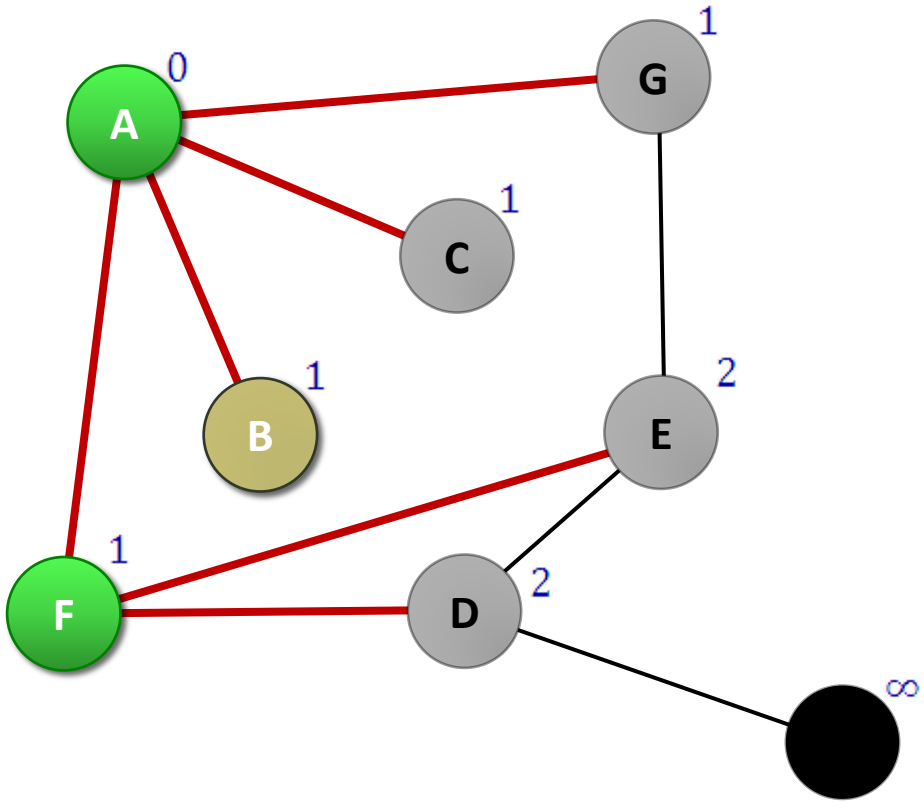


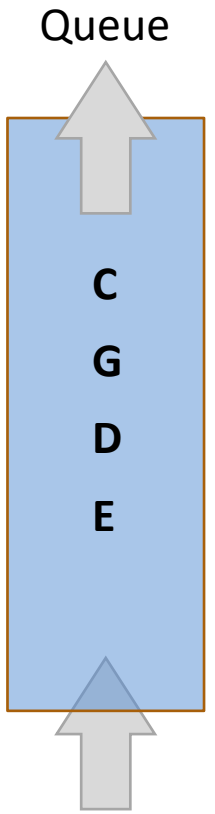
BFS



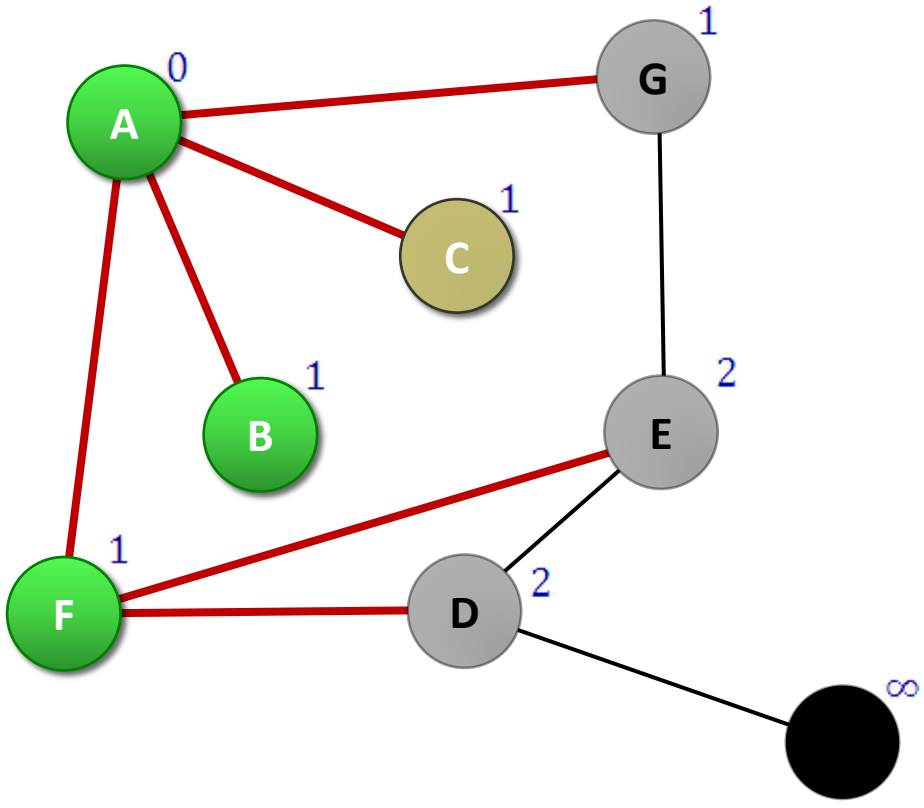


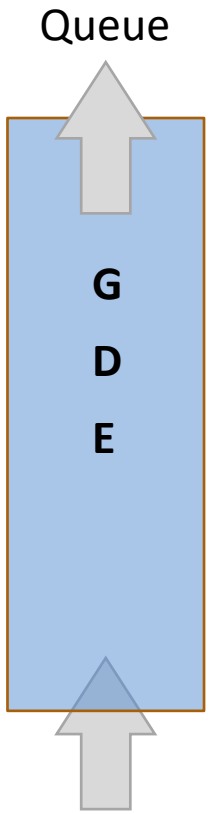
BFS



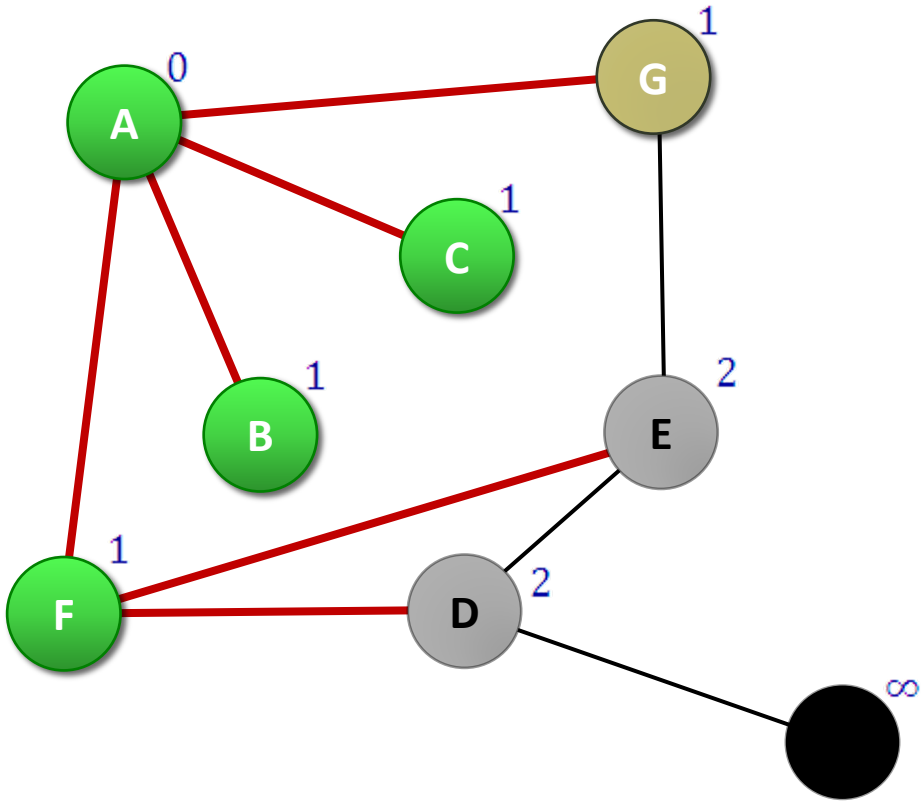


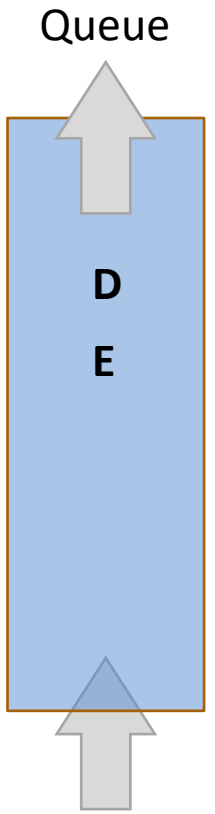
BFS



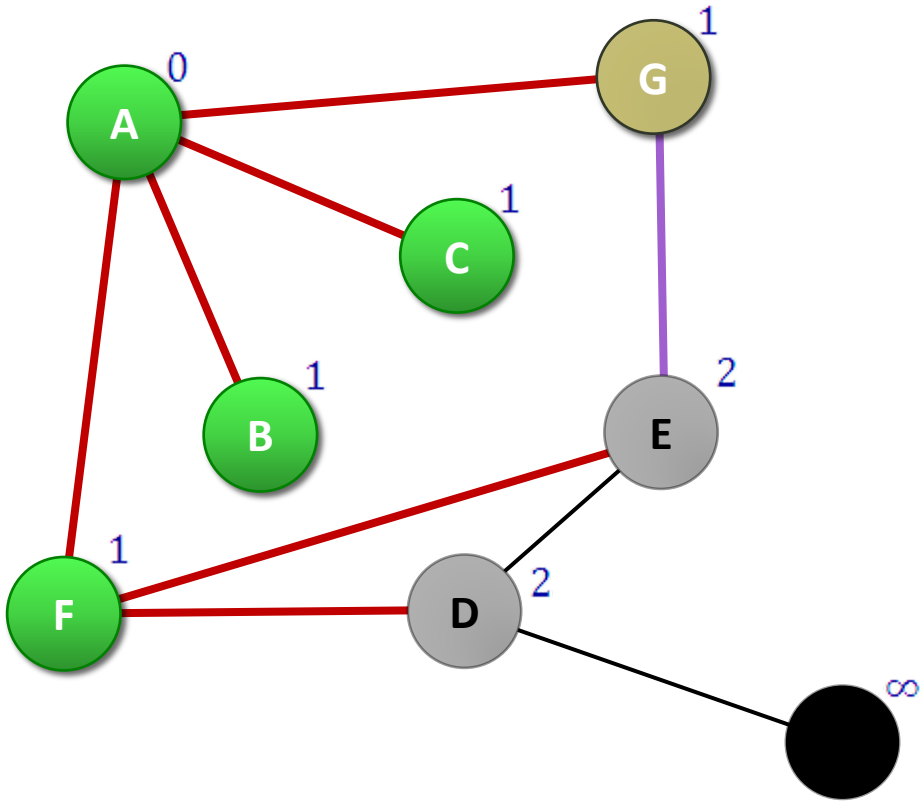


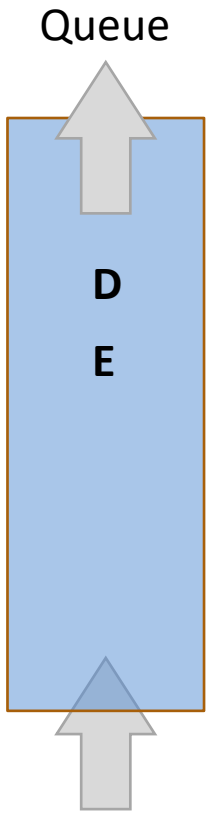
BFS



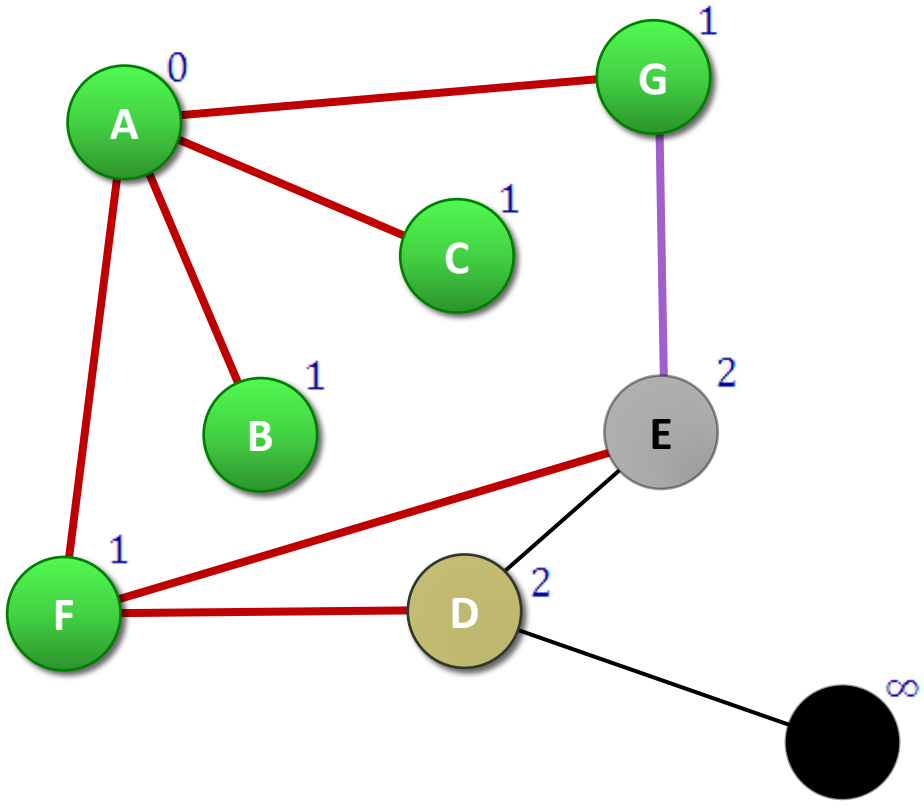


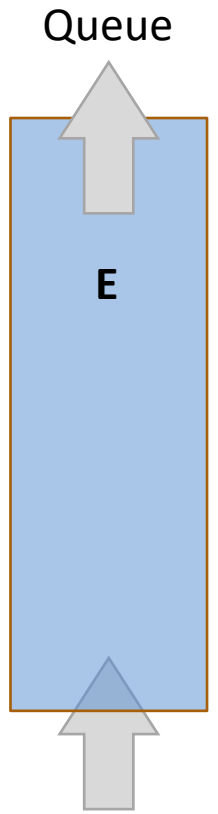
BFS



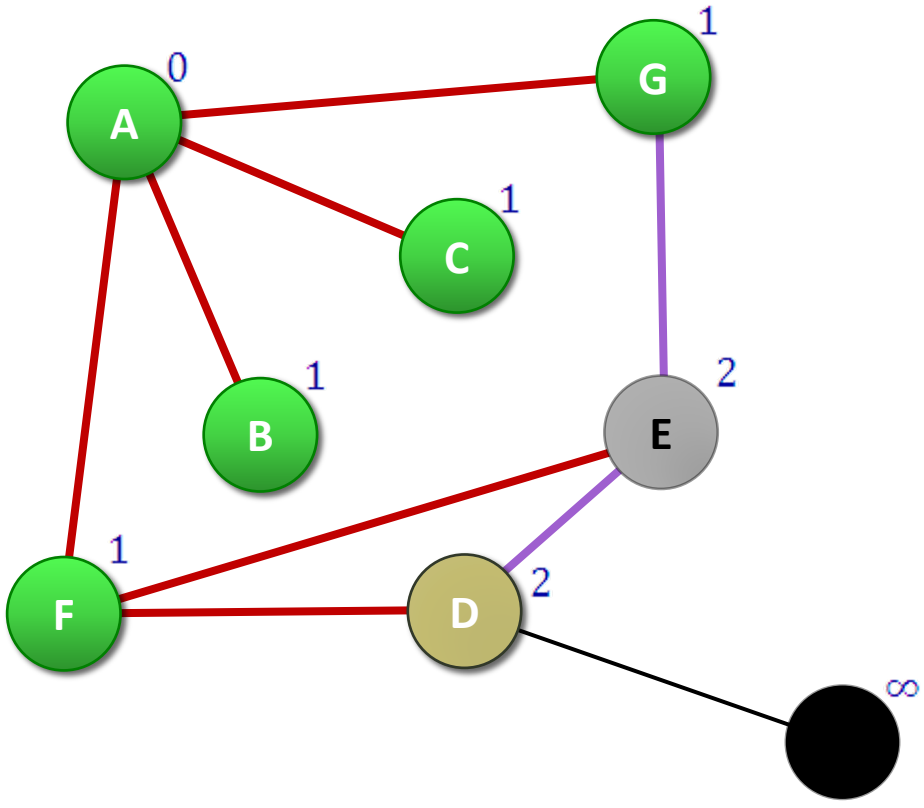


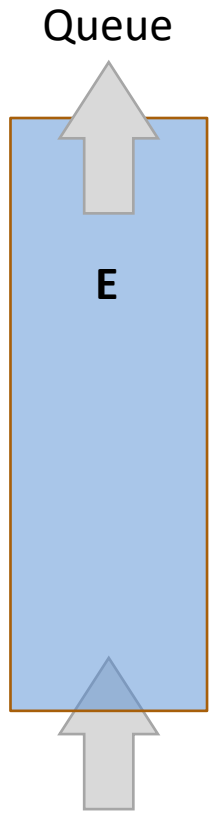
BFS



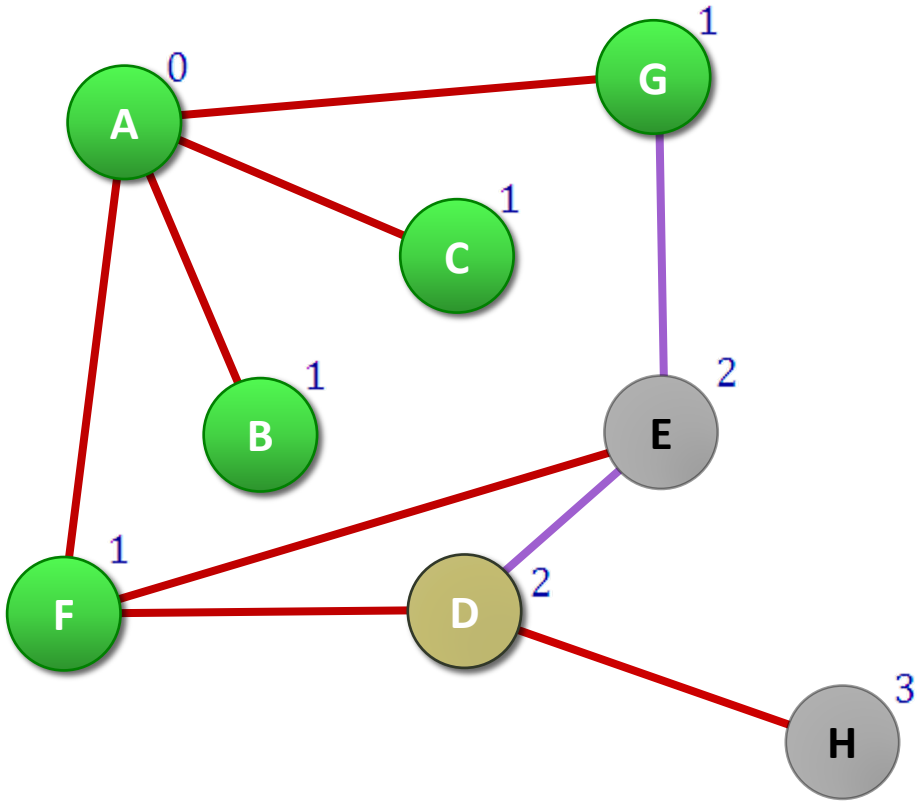


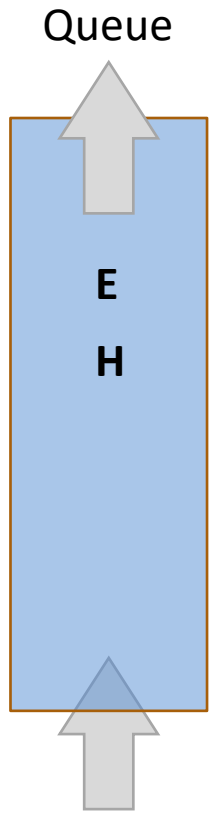
BFS



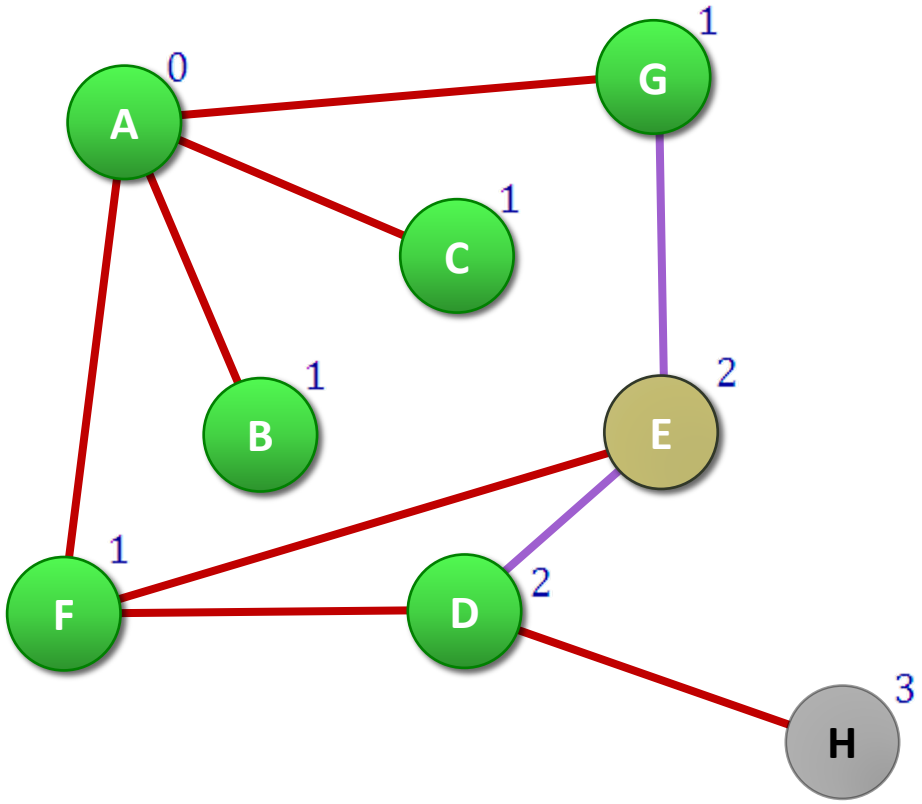


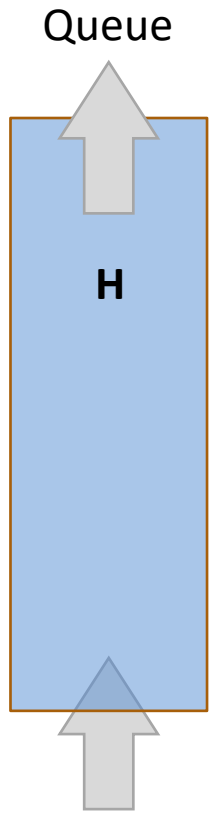
BFS



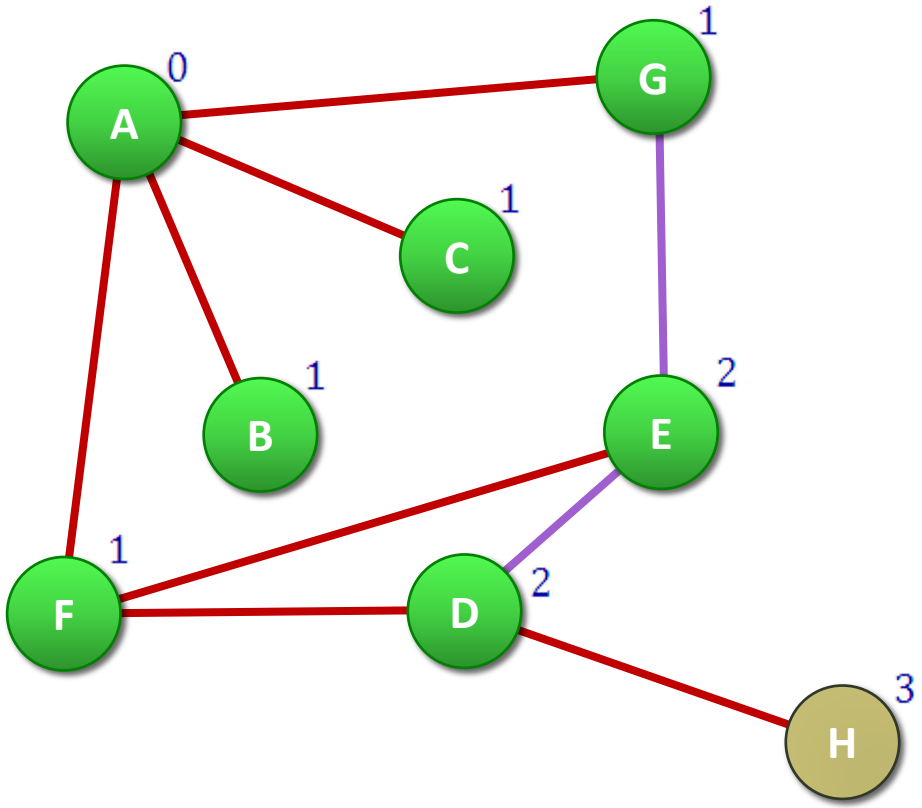


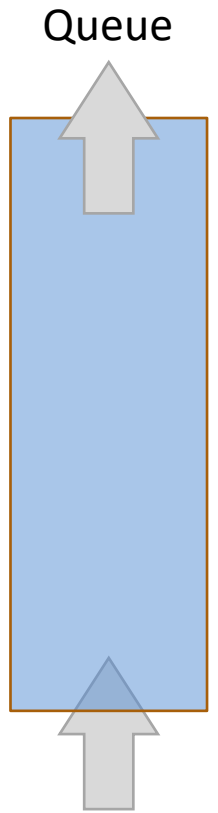
BFS



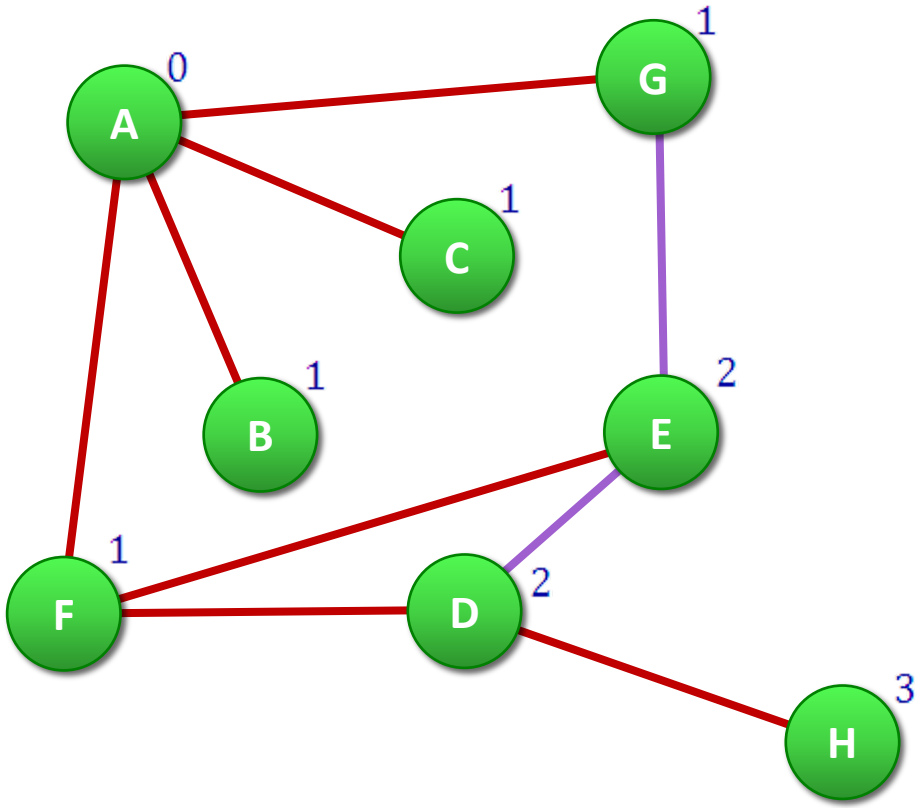


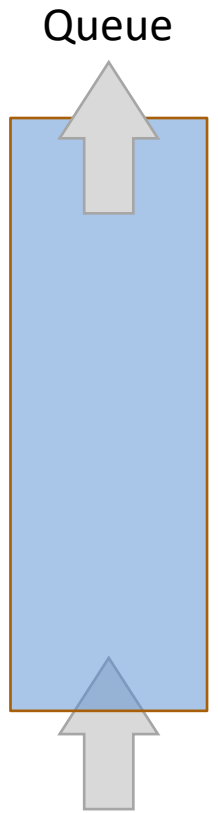
BFS



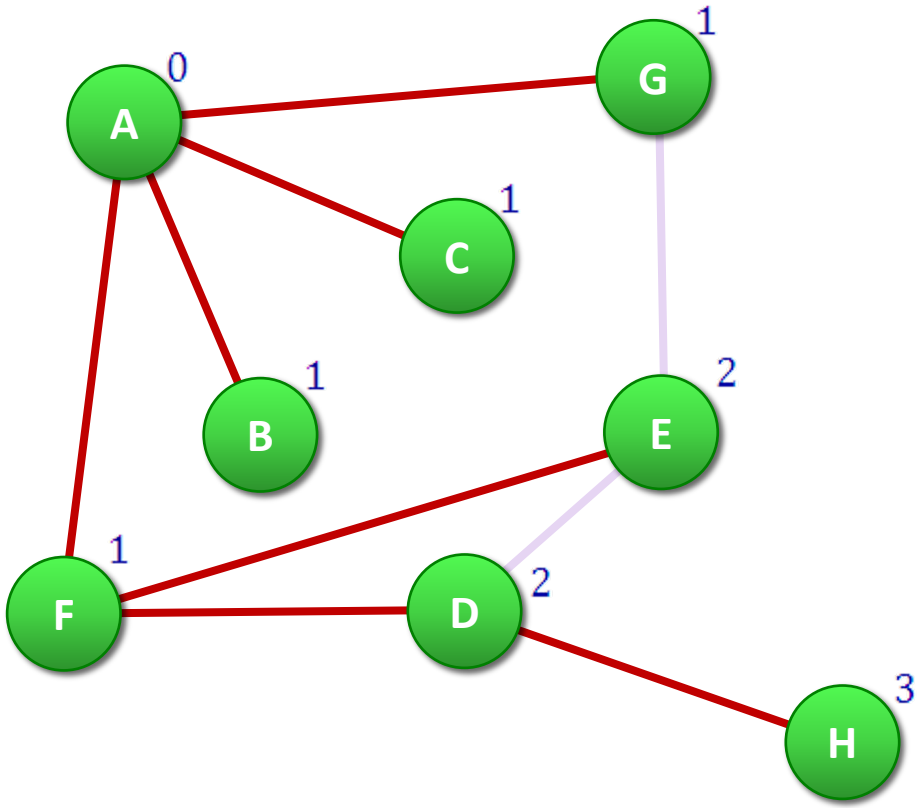


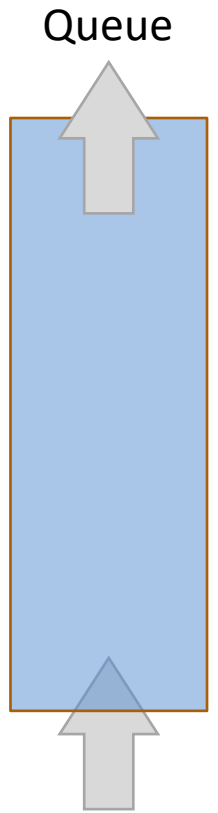
BFS





BFS – Resulting Tree





Exercise 2

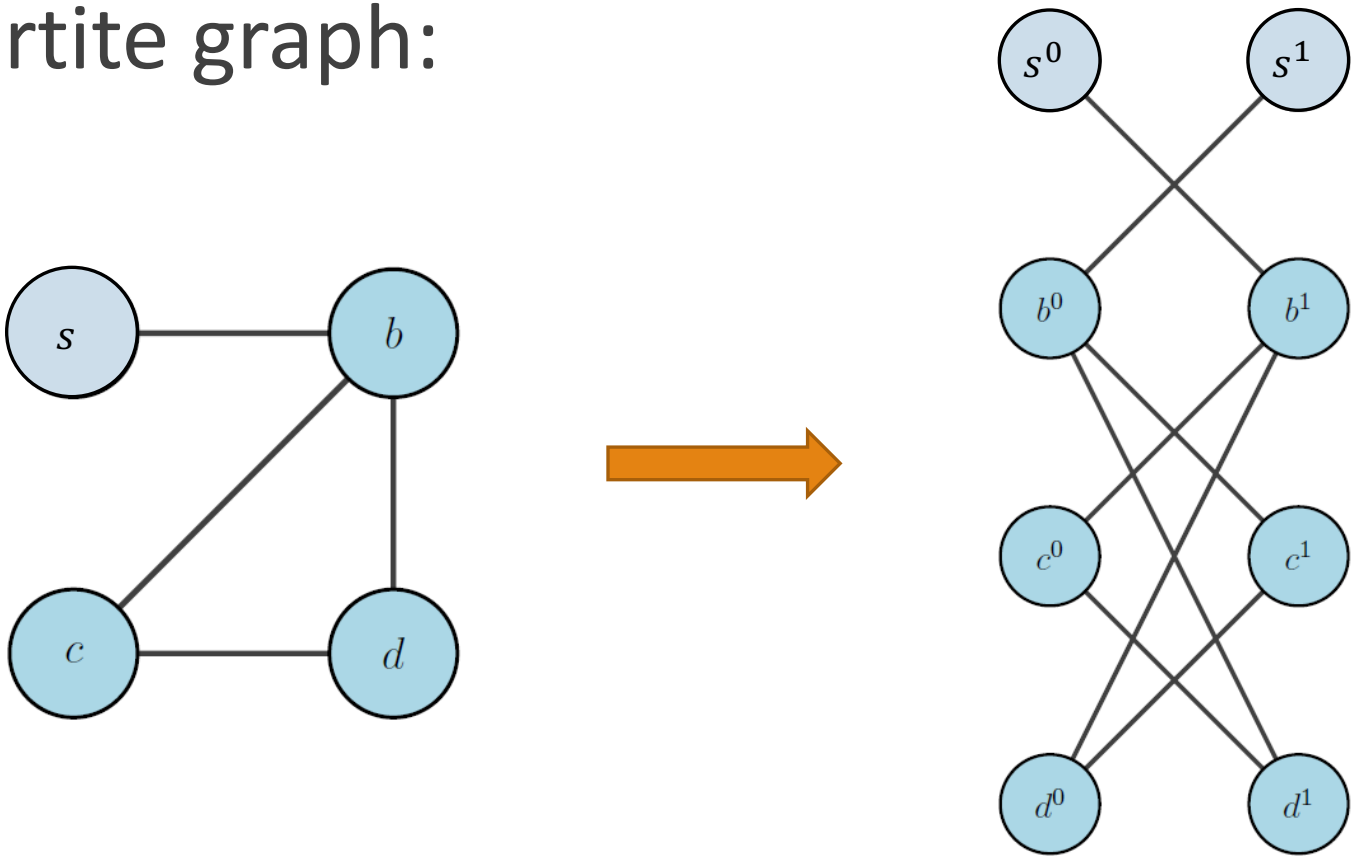
Let G(V, E) be an undirected graph, and let $s \in V$ be some vertex in G. Suggest a linear algorithm that finds for every vertex $v \in V$, the shortest path from s to v, consisting of an even number of edges.

Exercise 2 - Solution

- Intuition: we want to use BFS, but need to do some changes.
- **Important: if we make changes in BFS, we need to prove correctness and analyze complexity again!**
- **Therefore, we try to avoid modifying algorithms.**
- Instead of changing the algorithm, we can change the input to the BFS algorithm.
- Reduction: solve a different problem and prove that solving the reduced problem is equivalent to solving the original problem.

Exercise 2 - Solution

Build a bipartite graph:



Exercise 2 - Solution

Algorithm:

1. Build the following bipartite graph: G' = (V', E') such that:
   - For all $v \in V$ we make two copies in G': $v^0, v^1 \in V'$
   - For all $(u, v) \in E$ we make two copies in G':
     - $(u^0, v^1) \in E'$
     - $(u^1, v^0) \in E'$
2. We perform $BFS(G', s^0)$ and obtain $d(s^0, v)$ for all $v \in V'$.
3. For all $v \in V$ we return $d(s^0, v^0)$.

Exercise 2 - Solution

Correctness:

G' is built such that every path from V^i to V^{1-i}, $i \in \{0,1\}$, corresponds to an odd path in G, and every path from V^i to V^i, $i \in \{0,1\}$, corresponds to an even path in G, and vice versa. This is true since all edges connect different sides of a bipartite graph.

Also, every path in G corresponds to a path in G' that alternates between V^0 and V^1.

Exercise 2 - Solution

Correctness (continued):

To show that those problems are equivalent, need to show that every path of even length in G exists with the same length in G' and vice versa.

Claim: There exists a path p of length 2k from s to v in G if and only if there exists a path p' of length 2k from s^0 to v^0 in G'.

Exercise 2 - Solution

Correctness (continued):

Claim: There exists a path p of length 2k from s to v in G if and only if there exists a path p' of length 2k from s^0 to v^0 in G'.

Proof: ➔ Let p be some path of even length in G:

$p = (v_0, v_1, \dots, v_{2k})$. From the construction of G' we know there exists some path: $p' = (v_0^0, v_1^1, \dots, v_{2k}^0)$ in G'. Notice that the length does not change, and every edge from some v_i to v_{i+1} is present from V^1 to V^0 and vice versa.

➔ Similar.

Exercise 2 - Solution

Complexity:

1. Building the graph: n'=2n, m'= 2m ➔ O(m + n)
2. Performing BFS on G' - O(n'+m') = O(m + n)
3. Returning $d(s^0, v^0)$ - O(1) for every v∈V ➔ O(n)

All together: O(n+m)